

LeetCode Hot 100 全量题解手册

Go + Python · 100 题逐题图解 · 60 天复习路径

每题固定结构：题意抽象 → 核心观察 → 不变量/状态 → 手推图 → 算法步骤 → 双语言实现 → 复杂度 → 易错点 → 扩展

目标不是背 100 份代码，而是把 100 题压缩为可迁移的模式，并能在 Go / Python 间快速切换。

哈希	双指针	滑动窗口	前缀和	单调队列	数组/DP
数组	矩阵	链表	链表/堆	链表/哈希	二叉树
二叉树/前缀和	图	图/BFS	图/拓扑	Trie	回溯
二分查找	栈	栈/单调栈	堆	贪心	动态规划
多维动态规划	技巧				

模式地图：看到什么信号，就想到什么算法

哈希

#1 两数之和、#49 字母异位词分组、#128 最长连续序列

双指针

#283 移动零、#11 盛最多水的容器、#15 三数之和、#42 接雨水

滑动窗口

#3 无重复字符的最长子串、#438 找到字符串中所有字母异位词、#76 最小覆盖子串

前缀和

#560 和为 K 的子数组

单调队列

#239 滑动窗口最大值

数组/DP

#53 最大子数组和

数组

#56 合并区间、#189 轮转数组、#238 除自身以外数组的乘积、#41 缺失的第一个正数

矩阵

#73 矩阵置零、#54 螺旋矩阵、#48 旋转图像、#240 搜索二维矩阵 II

链表

#160 相交链表、#206 反转链表、#234 回文链表、#141 环形链表、#142 环形链表 II、#21 合并两个有序链表、#2 两数相加、#19 删除链表的倒数第 N 个结点、#24 两两交换链表中的节点、#25 K 个一组翻转链表、#138 随机链表的复制、#148 排序链表

模式地图：看到什么信号，就想到什么算法

链表/堆

#23 合并 K 个升序链表

链表/哈希

#146 LRU 缓存

二叉树

#94 二叉树的中序遍历、#104 二叉树的最大深度、#226 翻转二叉树、#101 对称二叉树、#543 二叉树的直径、#102 二叉树的层序遍历、#108 将有序数组转换为二叉搜索树、#98 验证二叉搜索树、#230 二叉搜索树中第 K 小的元素、#199 二叉树的右视图、#114 二叉树展开为链表、#105 从前序与中序遍历序列构造二叉树、#236 二叉树的最近公共祖先、#124 二叉树中的最大路径和

二叉树/前缀和

#437 路径总和 III

图

#200 岛屿数量

图/BFS

#994 腐烂的橘子

图/拓扑

#207 课程表

Trie

#208 实现 Trie (前缀树)

回溯

#46 全排列、#78 子集、#17 电话号码的字母组合、#39 组合总和、#22 括号生成、#79 单词搜索、#131 分割回文串、#51 N 皇后

模式地图：看到什么信号，就想到什么算法

二分查找

#35 搜索插入位置、#74 搜索二维矩阵、#34 在排序数组中查找元素的第一个和最后一个位置、#33 搜索旋转排序数组、#153 寻找旋转排序数组中的最小值、#4 寻找两个正序数组的中位数

栈

#20 有效的括号、#155 最小栈、#394 字符串解码

栈/单调栈

#739 每日温度、#84 柱状图中最大的矩形

堆

#215 数组中的第 K 个最大元素、#347 前 K 个高频元素、#295 数据流的中位数

贪心

#121 买卖股票的最佳时机、#55 跳跃游戏、#45 跳跃游戏 II、#763 划分子母区间

动态规划

#70 爬楼梯、#118 杨辉三角、#198 打家劫舍、#279 完全平方数、#322 零钱兑换、#139 单词拆分、#300 最长递增子序列、#152 乘积最大子数组、#416 分割等和子集、#32 最长有效括号

多维动态规划

#62 不同路径、#64 最小路径和、#5 最长回文子串、#1143 最长公共子序列、#72 编辑距离

技巧

#136 只出现一次的数字、#169 多数元素、#75 颜色分类、#31 下一个排列、#287 寻找重复数

核心观察

一遍扫描：在写入当前值之前，先查询 $\text{target}-x$ 是否已经出现。

复杂度

$O(n)$ time / $O(n)$ space

为什么这个算法成立

关键不变量：建立 $\text{value} \rightarrow \text{index}$ 哈希。随后通过“遍历 $x \rightarrow$ 查询 $\text{target}-x \rightarrow$ 命中立即返回，否则写入 x ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

先查后写，避免同一元素被使用两次。

1. 建立 $\text{value} \rightarrow \text{index}$ 哈希



2. 遍历 x



3. 查询 $\text{target}-x$



4. 命中立即返回，否则写入 x

题目示意图

输入

状态/不变量

核心操作

结果

$[2, 7, 11, 15]$, $\text{target}=9$: 2 入表后，扫描到 7 时命中 $\text{complement}=2$

扩展

Two Sum II、3Sum、kSum、两数差值问题

Go

```
func twoSum(nums []int, target int) []int {
    pos := map[int]int{}
    for i, x := range nums {
        if j, ok := pos[target-x]; ok { return []int{j, i} }
        pos[x] = i
    }
    return nil
}
```

Python

```
def twoSum(nums, target):
    pos = {}
    for i, x in enumerate(nums):
        if target - x in pos:
            return [pos[target-x], i]
        pos[x] = i
    return []
```

核心观察

给每个字符串构造稳定签名：排序后的字符串，或 26 字母计数向量。相同签名进入同一组。

复杂度

$O(n*k)$ 使用计数签名 / $O(n*k \log k)$ 使用排序

为什么这个算法成立

关键不变量：遍历字符串。随后通过“生成频次签名 $\rightarrow$ 签名作为 map key $\rightarrow$ 追加到对应分组”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

Python list 不能作为 dict key; Go 的 `[26]int` 可以作为 map key。

1. 遍历字符串



2. 生成频次签名



3. 签名作为 map key



4. 追加到对应分组

题目示意图

输入

状态/不变量

核心操作

结果

eat $\rightarrow$ (1a,1e,1t), tea 得到相同签名，因此落入同一 bucket

扩展

更大字符集时改用排序签名或稀疏频次

Go

```
func groupAnagrams(strs []string) [][]string {
    m := map[[26]int][]string{}
    for _, s := range strs {
        var k [26]int
        for i:=0;i<len(s);i++ { k[s[i]-'a']++ }
        m[k] = append(m[k], s)
    }
    ans := [][]string{}
    for _, g := range m { ans = append(ans, g) }
    return ans
}
```

Python

```
def groupAnagrams(strs):
    groups = {}
    for s in strs:
        cnt = [0] * 26
        for ch in s:
            cnt[ord(ch)-97] += 1
        groups.setdefault(tuple(cnt), []).append(s)
    return list(groups.values())
```

核心观察

把所有数字放入 `set`，只从不存在 `x-1` 的数字开始向右扩展；每段连续区间只会被扫描一次。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量：构造集合。随后通过“找序列起点 `x-1` 不存在 $\rightarrow$ 从 `x` 向右扩展 $\rightarrow$ 更新最长长度”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

若从每个元素都向右扩展，最坏退化到 $O(n^2)$ 。

1. 构造集合

2. 找序列起点 `x-1` 不存在3. 从 `x` 向右扩展

4. 更新最长长度

题目示意图

输入

状态/不变量

核心操作

结果

`[100,4,200,1,3,2]`：仅从 1 开始扩展得到 1-2-3-4

扩展

并查集、区间合并、流式连续段维护

Go

```
func longestConsecutive(nums []int) int {
    st:=map[int]bool{}; for _,x:=range nums { st[x]=true }
    best:=0
    for x:=range st {
        if !st[x-1] { y:=x; for st[y] { y++ }; if y-x>best { best=y-x } }
    }
    return best
}
```

Python

```
def longestConsecutive(nums):
    st = set(nums)
    best = 0
    for x in st:
        if x - 1 not in st:
            y = x
            while y in st:
                y += 1
            best = max(best, y-x)
    return best
```

核心观察

`slow` 表示下一个非零元素应写入的位置，`fast` 扫描数组；遇非零就写到 `slow` 并推进。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：`slow=0`。随后通过“`fast` 扫描 → 非零写到 `slow` → 尾部补 0”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

必须保持非零元素相对顺序；覆盖法最后需要补零。

1. `slow=0`



2. `fast` 扫描



3. 非零写到 `slow`



4. 尾部补 0

题目示意图



`0,1,0,3,12 -> 1,3,12,0,0`; `slow` 始终指向“下一个写入槽位”

扩展

移除元素、原地稳定压缩、颜色分类

Go

```
func moveZeroes(nums []int) {  
    slow:=0  
    for fast:=range nums {  
        if nums[fast]!=0 { nums[slow],nums[fast]=nums[fast],nums[slow];  
        slow++ }  
    }  
}
```

Python

```
def moveZeroes(nums):  
    slow = 0  
    for fast in range(len(nums)):  
        if nums[fast] != 0:  
            nums[slow], nums[fast] = nums[fast], nums[slow]  
            slow += 1
```

核心观察

左右指针形成容器；面积受较短边限制，因此每轮只能移动较短边，才有机会提高高度上限。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量： $l=0, r=n-1$ 。随后通过“计算面积 → 更新答案 → 移动较短边”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

需要能解释为何移动长边不可能让当前宽度下获得更大面积。

1. $l=0, r=n-1$



2. 计算面积



3. 更新答案



4. 移动较短边

题目示意图



l ----- width ----- r ; $height = \min(h[l], h[r])$, 只移动短板

扩展

双指针贪心证明、接雨水

Go

```
func maxArea(h []int) int {
    l, r, ans := 0, len(h)-1, 0
    for l < r {
        height := h[l]; if h[r] < height { height = h[r] }
        if a := height * (r - l); a > ans { ans = a }
        if h[l] < h[r] { l++ } else { r-- }
    }
    return ans
}
```

Python

```
def maxArea(height):
    l, r, ans = 0, len(height)-1, 0
    while l < r:
        ans = max(ans, min(height[l], height[r]) * (r-l))
        if height[l] < height[r]: l += 1
        else: r -= 1
    return ans
```

核心观察

排序后枚举第一个数 i ，把问题降为有序数组中的 Two Sum；移动 l/r ，并在三个层次上去重。

复杂度

$O(n^2)$ / $O(1)$ excluding output

为什么这个算法成立

关键不变量：排序。随后通过“枚举 $i \rightarrow$ 左右夹逼找 $-\text{nums}[i] \rightarrow$ 命中后左右去重”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

i 、 $left$ 、 $right$ 都要正确去重；超过 0 可提前结束。



题目示意图



固定 -1 后，在右侧区间用 l/r 寻找和为 1 的两数

扩展

4Sum、kSum 递归降维

Go

```
func threeSum(nums []int) [][]int {
    sort.Ints(nums); ans:=[][]int{}
    for i:=0;i<len(nums)-2;i++ {
        if i>0 && nums[i]==nums[i-1] { continue }; if nums[i]>0 { break }
        l,r:=i+1,len(nums)-1
        for l<r {
            s:=nums[i]+nums[l]+nums[r]
            if s<0 { l++ } else if s>0 { r-- } else {
                ans=append(ans,[]int{nums[i],nums[l],nums[r]}); l++; r--
                for l<r && nums[l]==nums[l-1] { l++ }; for l<r && nums[r]==nums[r+1] { r-- }
            }
        }
    }
    return ans
}
```

Python

```
def threeSum(nums):
    nums.sort(); ans = []
    for i in range(len(nums)-2):
        if i and nums[i] == nums[i-1]: continue
        if nums[i] > 0: break
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i]+nums[l]+nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                ans.append([nums[i],nums[l],nums[r]])
                l += 1; r -= 1
                while l < r and nums[l] == nums[l-1]: l += 1
                while l < r and nums[r] == nums[r+1]: r -= 1
    return ans
```

核心观察

维护 $\text{leftMax}/\text{rightMax}$ 。较小一侧的最高墙决定该侧当前格子的水位，因此先结算较小一侧。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：维护 $\text{leftMax}/\text{rightMax}$ 。随后通过“比较 $\text{leftMax}/\text{rightMax}$ → 结算较小侧水量 → 移动对应指针”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

先更新 max 再结算水量，且要理解为何另一侧至少存在不低于当前约束的墙。

1. 维护 $\text{leftMax}/\text{rightMax}$ 与两侧最大值



2. 比较 $\text{leftMax}/\text{rightMax}$



3. 结算较小侧水量



4. 移动对应指针

题目示意图



$\text{water}[i] = \min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$; 双指针在线计算

扩展

前后缀最大值、单调栈解法

Go

```
func trap(h []int) int {
    l, r, lm, rm, ans := 0, len(h)-1, 0, 0, 0
    for l <= r {
        if lm <= rm { if h[l] > lm { lm = h[l] }; ans += lm - h[l]; l++ }
        } else { if h[r] > rm { rm = h[r] }; ans += rm - h[r]; r-- }
    }
    return ans
}
```

Python

```
def trap(height):
    l, r = 0, len(height)-1
    lm = rm = ans = 0
    while l <= r:
        if lm <= rm:
            lm = max(lm, height[l]); ans += lm - height[l]; l += 1
        else:
            rm = max(rm, height[r]); ans += rm - height[r]; r -= 1
    return ans
```

核心观察

用 `lastPos` 或字符计数维护窗口。若字符重复，将 `left` 跳到 `max(left, last[ch]+1)`。

复杂度

$O(n)$ / $O(\text{字符集})$

为什么这个算法成立

关键不变量: `right` 扩张。随后通过“检查重复 $\rightarrow$ 必要时右移 `left` $\rightarrow$ 更新最大窗口”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

`left` 只能右移，不能因为旧位置较小而回退。

1. `right` 扩张



2. 检查重复

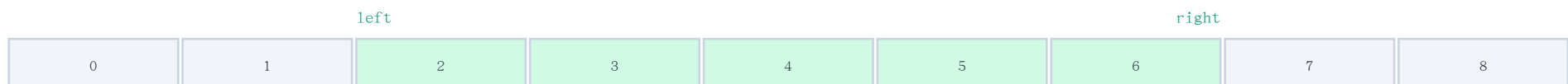


3. 必要时右移 `left`



4. 更新最大窗口

题目示意图



abcabcbb: 遇第二个 `a`, `left` 从 0 跳到 1; 窗口重新合法

扩展

最多 `K` 种字符、至多 `K` 次替换

Go

```
func lengthOfLongestSubstring(s string) int {  
    last:=map[byte]int{}; left,ans:=0,0  
    for r:=0;r<len(s);r++ {  
        if p,ok:=last[s[r]]; ok && p>=left { left=p+1 }  
        last[s[r]]=r; if r-left+1>ans { ans=r-left+1 }  
    }  
    return ans  
}
```

Python

```
def lengthOfLongestSubstring(s):  
    last = {}; left = ans = 0  
    for right, ch in enumerate(s):  
        if ch in last:  
            left = max(left, last[ch]+1)  
        last[ch] = right  
        ans = max(ans, right-left+1)  
    return ans
```

核心观察

窗口长度固定为 $\text{len}(p)$ ，维护 26 个字符频次差或 need/window 计数。

复杂度

$O(n)$ / $O(\text{字符集})$

为什么这个算法成立

关键不变量：统计 p 频次。随后通过“扩张 $\text{right} \rightarrow$ 窗口过长移 $\text{left} \rightarrow$ 匹配时记录 left ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

不要对每个窗口重新排序；窗口长度必须固定。

1. 统计 p 频次



2. 扩张 right

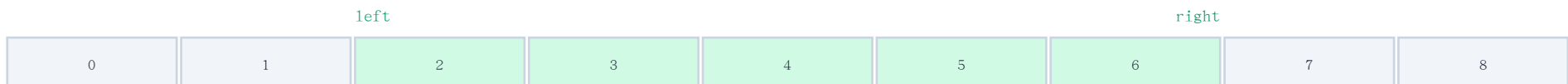


3. 窗口过长移 left



4. 匹配时记录 left

题目示意图



cbaebabacd / abc: 窗口 cba 与 bacd 中 abc 频次匹配时记录下标

扩展

Permutation in String、固定窗口统计

Go

```
func findAnagrams(s,p string) []int {
    if len(p)>len(s) { return nil }; var need,win [26]int
    for i:=range p { need[p[i]-'a']++ }; ans:=[]int{}; k:=len(p)
    for i:=range s {
        win[s[i]-'a']++
        if i>=k { win[s[i-k]-'a']-- }
        if i>=k-1 && win==need { ans=append(ans,i-k+1) }
    }
    return ans
}
```

Python

```
def findAnagrams(s, p):
    if len(p) > len(s): return []
    need = [0]*26; win = [0]*26
    for ch in p: need[ord(ch)-97] += 1
    ans = []; k = len(p)
    for i, ch in enumerate(s):
        win[ord(ch)-97] += 1
        if i >= k: win[ord(s[i-k])-97] -= 1
        if i >= k-1 and win == need: ans.append(i-k+1)
    return ans
```

核心观察

若当前前缀和为 s ，则历史前缀和 $s-k$ 出现多少次，就有多少个以当前位置结尾的合法子数组。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量: $sum=0, cnt\{0:1\}$ 。随后通过“加入 $nums[i]$ → 累加 $cnt[sum-k]$ → 再记录当前 sum ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

必须初始化 $count[0]=1$ ，表示从下标 0 开始的子数组。

1. $sum=0, cnt\{0:1\}$



2. 加入 $nums[i]$

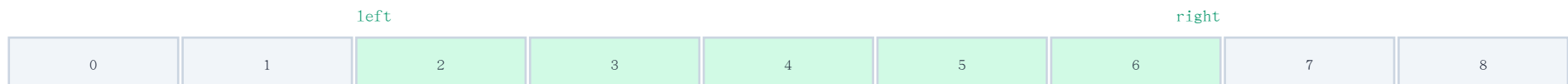


3. 累加 $cnt[sum-k]$



4. 再记录当前 sum

题目示意图



$prefix[j]-prefix[i]=k \Rightarrow$ 查历史 $prefix=当前-k$

扩展

和可被 K 整除、树路径前缀和、二维前缀和

Go

```
func subarraySum(nums []int, k int) int {  
    cnt:=map[int]int{0:1}; sum,ans:=0,0  
    for _,x:=range nums { sum+=x; ans+=cnt[sum-k]; cnt[sum]++ }  
    return ans  
}
```

Python

```
def subarraySum(nums, k):  
    cnt = {0:1}; s = ans = 0  
    for x in nums:  
        s += x  
        ans += cnt.get(s-k, 0)  
        cnt[s] = cnt.get(s, 0) + 1  
    return ans
```

核心观察

deque 存下标，且对应值单调递减；队头永远是当前窗口最大值。

复杂度

$O(n)$ / $O(k)$

为什么这个算法成立

关键不变量：移除过期队头。随后通过“弹出所有 $\leq$ 当前值的队尾 $\rightarrow$ 当前下标入队 $\rightarrow$ 窗口形成后取队头”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

队列必须存下标才能判断过期；每个元素最多进出队一次。

1. 移除过期队头



2. 弹出所有 $\leq$ 当前值的队尾



3. 当前下标入队



4. 窗口形成后取队头

题目示意图

输入

状态/不变量

核心操作

结果

$[1, 3, -1, -3, 5]$ ，deque 只保留仍可能成为未来最大值的候选下标

扩展

窗口最小值、单调队列优化 DP

Go

```
func maxSlidingWindow(nums []int, k int) []int {
    q:=[]int{}; ans:=[]int{}
    for i,x:=range nums {
        for len(q)>0 && q[0]<=i-k { q=q[1:] }
        for len(q)>0 && nums[q[len(q)-1]]<=x { q=q[:len(q)-1] }
        q=append(q,i); if i>=k-1 { ans=append(ans,nums[q[0]]) }
    }
    return ans
}
```

Python

```
from collections import deque
def maxSlidingWindow(nums, k):
    dq, ans = deque(), []
    for i, x in enumerate(nums):
        while dq and dq[0] <= i-k: dq.popleft()
        while dq and nums[dq[-1]] <= x: dq.pop()
        dq.append(i)
        if i >= k-1: ans.append(nums[dq[0]])
    return ans
```

核心观察

`right` 扩张直到窗口满足 `t` 的全部需求；然后不断收缩 `left`，同时维护最短合法窗口。

复杂度

$O(|s| + |t|)$ / $O(\text{字符集})$

为什么这个算法成立

关键不变量：建立 `need`。随后通过“扩张 `right` 更新 `window` → 全部满足后收缩 → 记录最短区间”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

`valid` 统计的是“满足需求的字符种类”，重复字符需要按频次精确处理。

1. 建立 `need`



2. 扩张 `right` 更新 `window`

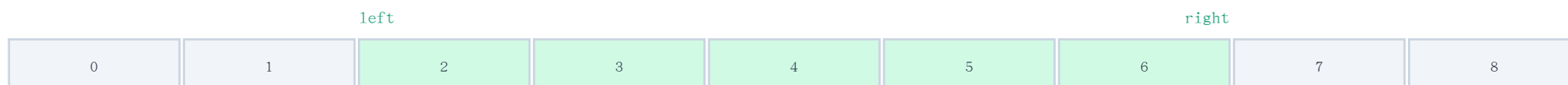


3. 全部满足后收缩



4. 记录最短区间

题目示意图



ADOBECODABANC: 第一次满足 ADOBEC 后继续搜索，最终收缩到 BANC

扩展

最短满足约束窗口、字符串包含排列

Go

```
func minWindow(s,t string) string {
    need:=map[byte]int{}; for i:=range t { need[t[i]]++ }
    win:=map[byte]int{}; valid,left,start,best:=0,0,0,len(s)+1
    for r:=0;r<len(s);r++ {
        c:=s[r]; win[c]++; if need[c]>0 && win[c]==need[c] { valid++ }
        for valid==len(need) {
            if r-left+1<best { start,best=left,r-left+1 }
            d:=s[left]; left++; if need[d]>0 && win[d]==need[d] { valid-- }
        }
    }
    if best==len(s)+1 { return "" }; return s[start:start+best]
}
```

Python

```
def minWindow(s, t):
    from collections import Counter
    need = Counter(t); missing = len(t)
    left = start = 0; best = float('inf')
    for right, ch in enumerate(s, 1):
        if need[ch] > 0: missing -= 1
        need[ch] -= 1
        if missing == 0:
            while left < right and need[s[left]] < 0:
                need[s[left]] += 1; left += 1
            if right-left < best: start, best = left, right-left
            need[s[left]] += 1; missing += 1; left += 1
    return '' if best == float('inf') else s[start:start+best]
```

核心观察

Kadane: dp 表示“必须以当前位置结尾”的最大和; $dp = \max(nums[i], dp + nums[i])$ 。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: 初始化 $cur = best = nums[0]$ 。随后通过“逐项更新 $cur \rightarrow best = \max(best, cur) \rightarrow$ 必要时记录起止点”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

全负数时不能把答案初始化为 0。

1. 初始化 $cur = best = nums[0]$



2. 逐项更新 cur



3. $best = \max(best, cur)$



4. 必要时记录起止点

题目示意图

输入

状态/不变量

核心操作

结果

若之前累加和为负, 则对当前元素只会有害, 因此从当前重新开始

扩展

返回最大区间、最大环形子数组、二维最大子矩阵

Go

```
func maxSubArray(nums []int) int {  
    cur, best := nums[0], nums[0]  
    for _, x := range nums[1:] { if cur+x > x { cur = cur+x } else { cur = x }; if cur > best { best = cur } }  
    return best  
}
```

Python

```
def maxSubArray(nums):  
    cur = best = nums[0]  
    for x in nums[1:]:  
        cur = max(x, cur+x)  
        best = max(best, cur)  
    return best
```

核心观察

按起点排序；若当前 $start \leq last.end$ 就合并，否则开始一个新区间。

复杂度

$O(n \log n)$ / $O(n)$

为什么这个算法成立

关键不变量：按 $start$ 排序。随后通过“ans 放第一个区间 $\rightarrow$ 逐个比较 $last.end \rightarrow$ 重叠则扩 end ，否则 $append$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

先排序；注意题目对端点相等是否算重叠。

1. 按 $start$ 排序



2. ans 放第一个区间



3. 逐个比较 $last.end$



4. 重叠则扩 end ，否则 $append$

题目示意图

输入

状态/不变量

核心操作

结果

$[1,3], [2,6] \rightarrow [1,6]$ ；因为 $2 \leq 3$

扩展

插入区间、会议室、扫描线

Go

```
func merge(intervals [][]int) [][]int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    ans := [][]int{}
    for _, in := range intervals {
        if len(ans) == 0 || in[0] > ans[len(ans)-1][1] { ans = append(ans, in) }
        else if in[1] > ans[len(ans)-1][1] { ans[len(ans)-1][1] = in[1] }
    }
    return ans
}
```

Python

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0]); ans = []
    for s, e in intervals:
        if not ans or s > ans[-1][1]: ans.append([s, e])
        else: ans[-1][1] = max(ans[-1][1], e)
    return ans
```

核心观察

右移 k 位等价于三次反转：整体反转、前 k 段反转、后 $n-k$ 段反转。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

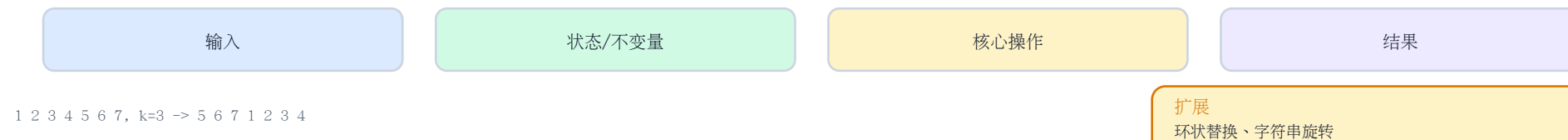
关键不变量: $k\%=n$ 。随后通过“reverse all $\rightarrow$ reverse $[0,k)$ $\rightarrow$ reverse $[k,n)$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

$k\%=n$; $n==0$ 时避免取模。



题目示意图



Go

```
func rotate(nums []int, k int) {  
    n := len(nums); if n == 0 {return}; k %= n  
    rev := func(a []int) {for i, j := 0, len(a) - 1; i < j; i, j = i + 1, j - 1 {a[i], a[j] = a[j], a[i]}}  
    rev(nums); rev(nums[:k]); rev(nums[k:])  
}
```

Python

```
def rotate(nums, k):  
    n = len(nums)  
    if n == 0: return  
    k %= n  
    nums[:] = nums[-k:] + nums[:-k] if k else nums
```

核心观察

`ans[i]` 先存左侧前缀积；再从右向左维护 `suffix` 并乘入 `ans[i]`。

复杂度

$O(n)$ / $O(1)$ extra excluding output

为什么这个算法成立

关键不变量：左到右写 `prefix`。随后通过“`suffix=1` → 右到左乘 `suffix` → 更新 `suffix`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

不要用除法；0 的情况前后缀法自然处理。

1. 左到右写 `prefix`



2. `suffix=1`



3. 右到左乘 `suffix`



4. 更新 `suffix`

题目示意图

输入

状态/不变量

核心操作

结果

```
ans[i]=product(nums[:i]) * product(nums[i+1:])
```

扩展

前后缀统计、乘法换成其他可结合运算

Go

```
func productExceptSelf(nums []int) []int {
    n:=len(nums); ans:=make([]int,n); p:=1
    for i:=0;i<n;i++ { ans[i]=p; p*=nums[i] }; s:=1
    for i:=n-1;i>=0;i-- { ans[i]*=s; s*=nums[i] }; return ans
}
```

Python

```
def productExceptSelf(nums):
    n=len(nums); ans=[1]*n; p=1
    for i in range(n): ans[i], p = p, p*nums[i]
    s=1
    for i in range(n-1,-1,-1):
        ans[i] *= s; s *= nums[i]
    return ans
```

核心观察

把 $1..n$ 范围内的值 x 原地交换到索引 $x-1$ ；最后第一个 $\text{nums}[i] \neq i+1$ 的位置就是答案。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：遍历 i 。随后通过“将合法 x 交换到 $x-1 \rightarrow$ 直到当前位置不能再放 $\rightarrow$ 二次扫描找首个错位”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

交换必须用 `while`；必须防止 $\text{nums}[x-1] == x$ 导致重复值死循环。

1. 遍历 i



2. 将合法 x 交换到 $x-1$



3. 直到当前位置不能再放



4. 二次扫描找首个错位

题目示意图

输入

状态/不变量

核心操作

结果

$[3, 4, -1, 1] \rightarrow$ 原地摆放后 $[1, -1, 3, 4]$ ，首个错位索引 $1 \Rightarrow 2$

扩展

原地哈希、缺失数、桶定位

Go

```
func firstMissingPositive(nums []int) int {
    n:=len(nums)
    for i:=0;i<n;i++ {
        for nums[i]>=1 && nums[i]<=n && nums[nums[i]-1]!=nums[i] { j:=nums[i]-1; nums[i],nums[j]=nums[j],nums[i] }
        }
    for i,x:=range nums { if x!=i+1 { return i+1 } }; return n+1
}
```

Python

```
def firstMissingPositive(nums):
    n=len(nums)
    for i in range(n):
        while 1 <= nums[i] <= n and nums[nums[i]-1] != nums[i]:
            j=nums[i]-1; nums[i], nums[j] = nums[j], nums[i]
    for i,x in enumerate(nums):
        if x != i+1: return i+1
    return n+1
```

核心观察

用第一行和第一列作为 $O(1)$ marker，并额外记录第一行/第一列本身是否包含 0。

复杂度

$O(mn)$ / $O(1)$

为什么这个算法成立

关键不变量：记录 firstRow/firstCol。随后通过“扫描内层并标记 → 根据 marker 清零内层 → 最后处理首行首列”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

先做标记，再从内层更新；最后处理首行首列。

1. 记录 firstRow/firstCol



2. 扫描内层并标记



3. 根据 marker 清零内层



4. 最后处理首行首列

题目示意图

输入

状态/不变量

核心操作

结果

某个 $\text{matrix}[i][j]=0 \Rightarrow$ 标记 $\text{matrix}[i][0]=0$ 与 $\text{matrix}[0][j]=0$

扩展

原地标记、状态压缩

Go

```
func setZeroes(a [][]int) {
    m,n:=len(a),len(a[0]); row0,col0:=false,false
    for j:=0;j<n;j++ { if a[0][j]==0 { row0=true } }; for i:=0;i<m;i++ {
        if a[i][0]==0 { col0=true } }
    for i:=1;i<m;i++ { for j:=1;j<n;j++ { if a[i][j]==0 { a[i][0]=0; a[0][j]=0 } } }
    for i:=1;i<m;i++ { for j:=1;j<n;j++ { if a[i][0]==0 || a[0][j]==0 { a[i][j]=0 } } }
    if row0 { for j:=0;j<n;j++ { a[0][j]=0 } }; if col0 { for i:=0;i<m;i++ { a[i][0]=0 } }
}
```

Python

```
def setZeroes(matrix):
    m,n=len(matrix),len(matrix[0])
    row0=any(matrix[0][j]==0 for j in range(n))
    col0=any(matrix[i][0]==0 for i in range(m))
    for i in range(1,m):
        for j in range(1,n):
            if matrix[i][j]==0: matrix[i][0]=matrix[0][j]=0
    for i in range(1,m):
        for j in range(1,n):
            if matrix[i][0]==0 or matrix[0][j]==0: matrix[i][j]=0
    if row0: matrix[0]=[0]*n
    if col0:
        for i in range(m): matrix[i][0]=0
```

核心观察

维护 top/bottom/left/right 四条边界，每次走一圈后向内收缩。

复杂度

$O(mn)$ / $O(1)$

为什么这个算法成立

关键不变量：走 top 从左到右。随后通过“走 right 从上到下 → 若仍合法走 bottom → 若仍合法走 left”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

走下边和左边之前必须再次检查边界，避免单行/单列重复。

1. 走 top 从左到右



2. 走 right 从上到下



3. 若仍合法走 bottom



4. 若仍合法走 left

题目示意图

输入

状态/不变量

核心操作

结果

四条边界不断向中心收缩: top++, right--, bottom--, left++

扩展
矩阵层序、螺旋填充

Go

```
func spiralOrder(a [][]int) []int {
    top,bottom,left,right:=0,len(a)-1,0,len(a[0])-1; ans:=[]int{}
    for top<=bottom && left<=right {
        for c:=left;c<=right;c++{ans=append(ans,a[top][c])}; top++
        for r:=top;r<=bottom;r++{ans=append(ans,a[r][right])}; right--
        if top<=bottom { for c:=right;c>=left;c--{ans=append(ans,a[bottom][c])}; bottom-- }
        if left<=right { for r:=bottom;r>=top;r--{ans=append(ans,a[r][left])}; left++ }
    }; return ans
}
```

Python

```
def spiralOrder(matrix):
    top,bottom,left,right=0,len(matrix)-1,0,len(matrix[0])-1; ans=[]
    while top<=bottom and left<=right:
        ans += matrix[top][left:right+1]; top += 1
        for r in range(top,bottom+1): ans.append(matrix[r][right])
        right -= 1
        if top<=bottom:
            ans += matrix[bottom][left:right+1][::-1]; bottom -= 1
        if left<=right:
            for r in range(bottom,top-1,-1): ans.append(matrix[r][left])
            left += 1
    return ans
```

核心观察

顺时针 90°：先沿主对角线转置，再把每一行反转。

复杂度

$O(n^2)$ / $O(1)$

为什么这个算法成立

关键不变量：transpose。随后通过“reverse each row → 完成顺时针旋转”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

必须原地；转置只交换 $j > i$ 的元素。

1. transpose



2. reverse each row



3. 完成顺时针旋转

题目示意图

输入

状态/不变量

核心操作

结果

`matrix[i][j] <=> matrix[j][i]`，然后每行 reverse

扩展

逆时针 90°、180° 旋转

Go

```
func rotate(matrix [][]int) {
    n:=len(matrix)
    for i:=0;i<n;i++ { for j:=i+1;j<n;j++ { matrix[i][j],matrix[j][i]=ma
trix[j][i],matrix[i][j] } }
    for i:=0;i<n;i++ { for l,r:=0,n-1;l<r;l,r=l+1,r-1 { matrix[i][l],mat
rix[i][r]=matrix[i][r],matrix[i][l] } }
}
```

Python

```
def rotate(matrix):
    n=len(matrix)
    for i in range(n):
        for j in range(i+1,n): matrix[i][j],matrix[j][i]=matrix[j][i],ma
trix[i][j]
    for row in matrix: row.reverse()
```

核心观察

从右上角开始：当前值太大就左移，太小就下移，每一步排除一整行或一整列。

复杂度

$O(m+n)$ / $O(1)$

为什么这个算法成立

关键不变量： $r=0, c=n-1$ 。随后通过“比较 $matrix[r][c] \rightarrow$ 大则 $c-- \rightarrow$ 小则 $r++$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

要利用“行递增 + 列递增”的二维单调性。

1. $r=0, c=n-1$



2. 比较 $matrix[r][c]$



3. 大则 $c--$



4. 小则 $r++$

题目示意图

输入

状态/不变量

核心操作

结果

从右上角看：向左只会变小，向下只会变大，因此能单调排除

扩展

Young tableau、左下角等价搜索

Go

```
func searchMatrix(a [][]int, target int) bool {
    r, c := 0, len(a[0]) - 1
    for r < len(a) && c >= 0 { if a[r][c] == target { return true }; if a[r][c] > target { c-- } else { r++ } }; return false
}
```

Python

```
def searchMatrix(matrix, target):
    r, c = 0, len(matrix[0]) - 1
    while r < len(matrix) and c >= 0:
        if matrix[r][c] == target: return True
        if matrix[r][c] > target: c -= 1
        else: r += 1
    return False
```

核心观察

A 指针走完 A 后切到 B, B 指针走完 B 后切到 A; 两条总路径长度相同, 因此交点处对齐。

复杂度

$O(m+n)$ / $O(1)$

为什么这个算法成立

关键不变量: $pA=headA, pB=headB$ 。随后通过“各自向前 → 到尾后切换链表 → $pA==pB$ 时返回”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

比较的是节点身份/地址, 不是节点值。

1. $pA=headA, pB=headB$



2. 各自向前



3. 到尾后切换链表



4. $pA==pB$ 时返回

题目示意图



A: $a1-a2-c1-c2$; B: $b1-b2-b3-c1-c2$; 换头后路径长度相等

扩展
长度差对齐法

Go

```
func getIntersectionNode(a,b *ListNode) *ListNode {  
    p,q:=a,b  
    for p!=q { if p==nil{p=b}else{p=p.Next}; if q==nil{q=a}else{q=q.Next}  
    } }  
    return p  
}
```

Python

```
def getIntersectionNode(headA, headB):  
    a,b=headA,headB  
    while a is not b:  
        a = a.next if a else headB  
        b = b.next if b else headA  
    return a
```

核心观察

prev/cur/next 三指针逐节点反转 next 方向。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: prev=nil。随后通过“保存 next $\rightarrow$ cur.next=prev $\rightarrow$ prev=cur;cur=next”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

修改 cur.next 之前必须保存 next。

1. prev=nil



2. 保存 next



3. cur.next=prev



4. prev=cur;cur=next

题目示意图



1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ nil 逐步变为 nil $\leftarrow$ 1 $\leftarrow$ 2 $\leftarrow$ 3

扩展

递归反转、区间反转、k 组反转

Go

```
func reverseList(head *ListNode) *ListNode {  
    var prev *ListNode  
    for cur:=head;cur!=nil; { next:=cur.Next; cur.Next=prev; prev=cur; cur=next }  
    return prev  
}
```

Python

```
def reverseList(head):  
    prev, cur = None, head  
    while cur:  
        nxt = cur.next  
        cur.next = prev  
        prev, cur = cur, nxt  
    return prev
```

核心观察

快慢指针找中点，反转后半段，再从头和后半段同时比较。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：快慢找中点。随后通过“反转后半段 → 双指针比较 → 可选恢复链表”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

奇偶长度边界；工程中可在比较后恢复链表。

1. 快慢找中点



2. 反转后半段



3. 双指针比较



4. 可选恢复链表

题目示意图



1->2->2->1: slow 到后半段起点，反转为 1->2 后与前半比较

扩展

链表中点、反转组合题

Go

```
func isPalindrome(head *ListNode) bool {
    slow, fast := head, head; for fast != nil && fast.Next != nil { slow = slow.Next
    ; fast = fast.Next.Next }
    var prev *ListNode; for slow != nil { n := slow.Next; slow.Next = prev; prev = slow; slow = n }
    for a, b := head, prev; b != nil; a, b = a.Next, b.Next { if a.Val != b.Val { return
    false } }; return true
}
```

Python

```
def isPalindrome(head):
    slow = fast = head
    while fast and fast.next: slow, fast = slow.next, fast.next.next
    prev = None
    while slow:
        nxt = slow.next; slow.next = prev; prev, slow = slow, nxt
    a, b = head, prev
    while b:
        if a.val != b.val: return False
        a, b = a.next, b.next
    return True
```

核心观察

Floyd 快慢指针: slow 每次 1 步, fast 每次 2 步; 若有环, 二者最终必相遇。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: $slow=head, fast=head$ 。随后通过 “ $slow+1 \ fast+2 \rightarrow$ 相遇 $\Rightarrow$ 有环 $\rightarrow fast$ 到 $nil \Rightarrow$ 无环” 不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

循环条件必须保证 $fast$ 与 $fast.next$ 非空。

1. $slow=head, fast=head$



2. $slow+1 \ fast+2$



3. 相遇 $\Rightarrow$ 有环



4. $fast$ 到 $nil \Rightarrow$ 无环

题目示意图



环内快指针每轮相对 slow 多走 1 步, 因此一定追上

扩展
环入口、重复数

Go

```
func hasCycle(head *ListNode) bool {
    slow, fast := head, head
    for fast != nil && fast.Next != nil { slow = slow.Next; fast = fast.Next.Next; }
    if slow == fast { return true }
    return false
}
```

Python

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    if slow is fast: return True
    return False
```

核心观察

快慢首次相遇后，一个指针回到 head；两者同时每次走一步，再次相遇的位置就是环入口。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：Floyd 找首次相遇。随后通过“ptr=head → ptr 与 slow 同速前进 → 再次相遇即入口”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

需要能解释距离关系： $\text{head} \rightarrow \text{entry} = \text{meeting} \rightarrow \text{entry} \bmod \text{cycle}$ 。

1. Floyd 找首次相遇



2. ptr=head



3. ptr 与 slow 同速前进



4. 再次相遇即入口

题目示意图



设入环前 a、相遇点距入口 b、环长 c，可由 $2(a+b)=a+b+kc$ 推导

扩展

Floyd 找重复数、求环长度

Go

```
func detectCycle(head *ListNode) *ListNode {
    slow, fast := head, head
    for fast != nil && fast.Next != nil {
        slow = slow.Next; fast = fast.Next.Next
        if slow == fast { p := head; for p != slow { p = p.Next; slow = slow.Next }; return p }
    }; return nil
}
```

Python

```
def detectCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            p = head
            while p is not slow: p, slow = p.next, slow.next
            return p
    return None
```

核心观察

`dummy + tail`; 每次把两个头节点中较小者接到 `tail` 后面。

复杂度

$O(m+n)$ / $O(1)$

为什么这个算法成立

关键不变量: `dummy/tail`。随后通过“比较 `l1, l2` → `tail` 接小者并推进 → 接上剩余链”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

循环结束后要把剩余链直接接上。

1. `dummy/tail`



2. 比较 `l1, l2`



3. `tail` 接小者并推进



4. 接上剩余链

题目示意图



`1->3->5` 与 `2->4->6`, `tail` 每次选择当前最小头节点

扩展

归并排序、合并 k 个链表

Go

```
func mergeTwoLists(a,b *ListNode) *ListNode {
    d:=&ListNode{};t:=d
    for a!=nil&&b!=nil { if a.Val<b.Val{t.Next=a;a=a.Next}else{t.Next=b;
b=b.Next};t=t.Next }
    if a!=nil{t.Next=a}else{t.Next=b};return d.Next
}
```

Python

```
def mergeTwoLists(a,b):
    dummy=tail=ListNode()
    while a and b:
        if a.val < b.val: tail.next,a=a,a.next
        else: tail.next,b=b,b.next
        tail=tail.next
    tail.next = a or b
    return dummy.next
```

核心观察

同步遍历两个逆序数字链表，逐位计算 $\text{sum} = x + y + \text{carry}$ ，并创建 $\text{sum} \% 10$ 节点。

复杂度

$O(\max(m, n))$ / $O(1)$ extra excluding output

为什么这个算法成立

关键不变量： $\text{carry} = 0$ 。随后通过“读取两边当前位 $\rightarrow$ sum 并更新 carry $\rightarrow$ 创建结果节点”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

循环条件包含 carry；链表长度可不同。

1. $\text{carry} = 0$



2. 读取两边当前位



3. sum 并更新 carry



4. 创建结果节点

题目示意图



$342 + 465$: $2+5=7$, $4+6=10$ 写0进1, $3+4+1=8 \rightarrow 807$

扩展

大整数加减乘、正序链表加法

Go

```
func addTwoNumbers(a,b *ListNode) *ListNode {
    d:=&ListNode{};t:=d;carry:=0
    for a!=nil||b!=nil||carry>0 { x,y:=0,0;if a!=nil{x=a.Val;a=a.Next};i
f b!=nil{y=b.Val;b=b.Next};s:=x+y+carry;carry=s/10;t.Next=&ListNode{Val:
s%10};t=t.Next }
    return d.Next
}
```

Python

```
def addTwoNumbers(l1,l2):
    dummy=tail=ListNode(); carry=0
    while l1 or l2 or carry:
        s=carry+(l1.val if l1 else 0)+(l2.val if l2 else 0)
        carry,d=divmod(s,10); tail.next=ListNode(d); tail=tail.next
        l1=l1.next if l1 else None; l2=l2.next if l2 else None
    return dummy.next
```

#19 删除链表的倒数第 N 个结点 / Remove Nth Node From End of List

链表 · Medium · 题解推导 + 易错点 + 扩展

核心观察

dummy; fast 先走 n 步, 然后 fast/slow 同速走到 fast 的下一步为空, 此时 slow.next 为待删节点。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: dummy->head。随后通过“fast 领先 n 步 → 同步移动 fast/slow → slow.next=slow.next.next”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

删除头节点时必须借助 dummy 统一处理。

1. dummy->head



2. fast 领先 n 步



3. 同步移动 fast/slow



4. slow.next=slow.next.next

题目示意图



保持 fast 与 slow 相距 n 个节点, fast 到尾时 slow 正好在待删前驱

扩展
固定距离双指针

Go

```
func removeNthFromEnd(head *ListNode, n int) *ListNode {
    d := &ListNode{Next: head}; fast, slow := d, d; for i := 0; i < n; i++ { fast = fast.Next }; for fast.Next != nil { fast = fast.Next; slow = slow.Next }; slow.Next = slow.Next.Next; return d.Next
}
```

Python

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head); fast = slow = dummy
    for _ in range(n): fast = fast.next
    while fast.next: fast, slow = fast.next, slow.next
    slow.next = slow.next.next
    return dummy.next
```

核心观察

每轮以前驱 `pre`、节点 `a`、`b` 三者重连: `pre->b`, `a->b.next`, `b->a`。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: `pre=dummy`。随后通过“`a=pre.next, b=a.next` → 重连三条边 → `pre=a`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

先保存 `b.next`; 每轮 `pre` 要移动到 `a`。

1. `pre=dummy`



2. `a=pre.next, b=a.next`



3. 重连三条边



4. `pre=a`

题目示意图



`pre -> a -> b -> next` 变成 `pre -> b -> a -> next`

扩展

`k` 个一组反转的简化版

Go

```
func swapPairs(head *ListNode) *ListNode {
    d:=&ListNode{Next:head};pre:=d
    for pre.Next!=nil&&pre.Next.Next!=nil { a,b:=pre.Next,pre.Next.Next;
a.Next=b.Next;b.Next=a;pre.Next=b;pre=a };return d.Next
}
```

Python

```
def swapPairs(head):
    dummy=ListNode(0,head); pre=dummy
    while pre.next and pre.next.next:
        a,b=pre.next,pre.next.next
        a.next=b.next; b.next=a; pre.next=b
        pre=a
    return dummy.next
```

核心观察

每组先找到第 k 个节点确认长度足够，再反转该闭区间并接回前后两段。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：定位 k th。随后通过“保存 $groupNext \rightarrow$ 反转 $groupNext \rightarrow$ 重新连接并推进 $groupPrev$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

不足 k 个必须保持原样； $groupPrev/groupNext$ 边界最易错。

1. 定位 k th



2. 保存 $groupNext$



3. 反转 $groupNext$



4. 重新连接并推进 $groupPrev$

题目示意图



每轮只翻转完整的 k 个节点块，尾部不足 k 的块不动

扩展
区间反转、链表分块

Go

```
func reverseKGroup(head *ListNode, k int) *ListNode {
    d := &ListNode{Next: head}; gp := d
    for { kth := gp; for i := 0; i < k; i++ { kth = kth.Next; if kth == nil { return d.Next } }; gn := kth.Next; prev, cur := gn, gp.Next
        for cur != gn { n := cur.Next; cur.Next = prev; prev = cur; cur = n }; old := gp.Next; gp.Next = kth; gp = old }
}
```

Python

```
def reverseKGroup(head, k):
    dummy = ListNode(0, head); gp = dummy
    while True:
        kth = gp
        for _ in range(k):
            kth = kth.next
        if not kth: return dummy.next
        gn = kth.next; prev, cur = gn, gp.next
        while cur is not gn:
            nxt = cur.next; cur.next = prev; prev, cur = cur, nxt
        old = gp.next; gp.next = kth; gp = old
```

核心观察

哈希做 old->new 映射：第一遍建所有副本，第二遍连接 next/random；也可原地穿插副本实现 $O(1)$ extra。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量：第一遍复制节点。随后通过“建立 old->new $\rightarrow$ 第二遍连 next/random $\rightarrow$ 返回 map[head]”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

random 可能为空；深拷贝不能让新节点指向旧节点。

1. 第一遍复制节点



2. 建立 old->new



3. 第二遍连 next/random



4. 返回 map[head]

题目示意图



old A random->C; newA.random 必须指向 newC, 而非旧 C

扩展

复杂图结构深拷贝、 $O(1)$ 穿插法

Go

```
func copyRandomList(head *Node) *Node {  
    mp:=map[*Node]*Node{nil:nil};for p:=head;p!=nil;p=p.Next{mp[p]=&Node  
    {Val:p.Val}}  
    for p:=head;p!=nil;p=p.Next{mp[p].Next=mp[p.Next];mp[p].Random=mp[p.  
    Random]};return mp[head]  
}
```

Python

```
def copyRandomList(head):  
    if not head: return None  
    mp={None:None}; cur=head  
    while cur: mp[cur]=Node(cur.val); cur=cur.next  
    cur=head  
    while cur:  
        mp[cur].next=mp[cur.next]; mp[cur].random=mp[cur.random]  
        cur=cur.next  
    return mp[head]
```

核心观察

链表最适合归并排序：快慢指针切半，递归排序两半，再线性 merge。

复杂度

$O(n \log n)$ / $O(\log n)$ recursion

为什么这个算法成立

关键不变量：快慢切半。随后通过“断开 left/right → 递归排序 → merge 两个有序链”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

切半时必须断链，否则递归不会缩小。

1. 快慢切半



2. 断开 left/right



3. 递归排序



4. merge 两个有序链

题目示意图



链表无需随机访问，merge 只改 next 指针，因此归并排序天然适配

扩展

自底向上归并把额外栈降到 $O(1)$

Go

```
func sortList(head *ListNode) *ListNode {
    if head==nil||head.Next==nil{return head};slow,fast:=head,head.Next;
    for fast!=nil&&fast.Next!=nil{slow=slow.Next;fast=fast.Next.Next};mid:=s
    low.Next;slow.Next=nil
    return mergeTwoLists(sortList(head),sortList(mid))
}
```

Python

```
def sortList(head):
    if not head or not head.next: return head
    slow,fast=head,head.next
    while fast and fast.next: slow,fast=slow.next,fast.next.next
    mid=slow.next; slow.next=None
    a,b=sortList(head),sortList(mid)
    dummy=tail=ListNode()
    while a and b:
        if a.val<b.val: tail.next,a=a,a.next
        else: tail.next,b=b,b.next
        tail=tail.next
    tail.next=a or b
    return dummy.next
```

核心观察

把每条链的当前头节点放入小顶堆；弹出最小节点后，把它的 next 放回堆。

复杂度

$O(N \log k)$ / $O(k)$

为什么这个算法成立

关键不变量：初始化 k 个头到堆。随后通过“pop 最小节点 → 接到结果链 → 其 next 非空则 push”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

Go heap.Interface 的 Less/Push/Pop 类型断言；Python 堆中相同 val 时需要 tie-break 或自定义对象可比性。

1. 初始化 k 个头到堆



2. pop 最小节点



3. 接到结果链



4. 其 next 非空则 push

题目示意图



堆始终只保存每条链“当前未消费的最小候选”

扩展

分治两两 merge、外部排序

Go

```
type NodeHeap []*ListNode
func (h NodeHeap) Len() int {return len(h)}; func (h NodeHeap) Less(i, j int) bool {return h[i].Val < h[j].Val}; func (h NodeHeap) Swap(i, j int) {h[i], h[j] = h[j], h[i]}
func (h *NodeHeap) Push(x any) {*h = append(*h, x.(*ListNode))}; func (h *NodeHeap) Pop() any {a := *h; x := a[len(a)-1]; *h = a[:len(a)-1]; return x}
func mergeKLists(lists []*ListNode) *ListNode {h := &NodeHeap{}; heap.Init(h); for _, n := range lists {if n != nil {heap.Push(h, n)}}; d := &ListNode{}; t := d; for h.Len() > 0 {n := heap.Pop(h).(*ListNode); t.Next = n; t = n; if n.Next != nil {heap.Push(h, n.Next)}}; return d.Next}
```

Python

```
import heapq
def mergeKLists(lists):
    h = []; seq = 0
    for node in lists:
        if node: heapq.heappush(h, (node.val, seq, node)); seq += 1
    dummy = tail = ListNode()
    while h:
        _, _, node = heapq.heappop(h); tail.next = node; tail = node
        if node.next: heapq.heappush(h, (node.next.val, seq, node.next)); seq += 1
    return dummy.next
```

核心观察

哈希 $O(1)$ 定位节点；双向链表按最近使用顺序排列，头部最新、尾部最旧。

复杂度

$O(1)$ get/put / $O(\text{capacity})$

为什么这个算法成立

关键不变量：get: map 定位。随后通过“命中则移动到头 $\rightarrow$ put: 新增/更新并移动到头 $\rightarrow$ 超容量删除尾部”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

更新已有 key 也要移动到头；淘汰尾节点时同步删除 map。

1. get: map 定位



2. 命中则移动到头



3. put: 新增/更新并移动到头



4. 超容量删除尾部

题目示意图



head $\leftrightarrow$ MRU ... LRU $\leftrightarrow$ tail; 哈希只负责定位，链表负责顺序

扩展

LFU、TTL cache、并发安全缓存

Go

```
type entry struct{key,val int;prev,next *entry}
type LRUCache struct{cap int;m map[int]*entry;head,tail *entry}
func Constructor(capacity int) LRUCache{h,t:=&entry{},&entry{};h.next=t;
t.prev=h;return LRUCache{capacity,map[int]*entry{},h,t}}
func(c *LRUCache)remove(n *entry){n.prev.next=n.next;n.next.prev=n.prev}
func(c *LRUCache)add(n *entry){n.next=c.head.next;n.prev=c.head;c.head.next.prev=n;
c.head.next=n}
func(c *LRUCache)Get(k int)int{ok:=c.m[k];if !ok{return -1};c.remove(n);c.add(n);return n.val}
func(c *LRUCache)Put(k,v int){if n,ok:=c.m[k];ok{c.remove(n)};n:=&entry{key:k,val:v};c.m[k]=n;c.add(n);if len(c.m)>c.cap{old:=c.tail.prev;c.remove(old);delete(c.m,old.key)}}
```

Python

```
class LRUCache:
    def __init__(self, capacity):
        self.cap=capacity; self.mp={}; self.head=Node(0,0); self.tail=None
    def __del__(self):
        self.head.next=self.tail; self.tail.prev=self.head
    def _remove(self,n): n.prev.next=n.next; n.next.prev=n.prev
    def _add(self,n):
        n.next=self.head.next; n.prev=self.head
        self.head.next.prev=n; self.head.next=n
    def get(self,key):
        if key not in self.mp: return -1
        n=self.mp[key]; self._remove(n); self._add(n); return n.val
    def put(self,key,val):
        if key in self.mp: self._remove(self.mp[key])
        n=Node(key,val); self.mp[key]=n; self._add(n)
        if len(self.mp)>self.cap:
            old=self.tail.prev; self._remove(old); del self.mp[old.key]
```

核心观察

递归顺序 left-root-right; 迭代时不断把左链压栈，弹栈访问，再转向右子树。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量：沿左链入栈。随后通过“弹出栈顶 → 访问节点 → 转向右子树”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

迭代模板中外层条件是 `cur!=nil || stack 非空`。

1. 沿左链入栈



2. 弹出栈顶

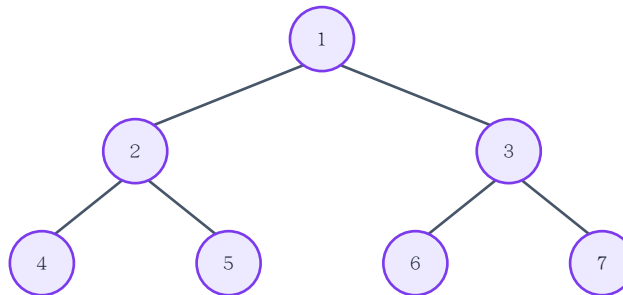


3. 访问节点



4. 转向右子树

题目示意图



BST 的中序遍历结果天然有序

扩展

前序/后序、Morris $O(1)$ 遍历

Go

```
func inorderTraversal(root *TreeNode) []int { ans:=[]int{};st:=[]*TreeNo
de{};cur:=root;for cur!=nil||len(st)>0{for cur!=nil{st=append(st,cur);cu
r=cur.Left};cur=st[len(st)-1];st=st[:len(st)-1];ans=append(ans,cur.Val);
cur=cur.Right};return ans }
```

Python

```
def inorderTraversal(root):
    ans, st, cur = [], [], root
    while cur or st:
        while cur: st.append(cur); cur=cur.left
        cur=st.pop(); ans.append(cur.val); cur=cur.right
    return ans
```

核心观察

DFS 返回 $1 + \max(\text{leftDepth}, \text{rightDepth})$; 或 BFS 逐层计数。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量：递归左右子树。随后通过“取更大深度 $\rightarrow +1$ 返回当前高度”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

空树深度为 0。

1. 递归左右子树

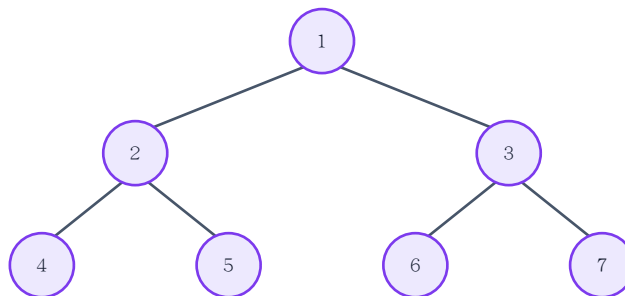


2. 取更大深度



3. +1 返回当前高度

题目示意图



叶子返回 1；父节点高度由左右子树高度的最大值决定

扩展

最小深度、树高、平衡判断

Go

```
func maxDepth(root *TreeNode) int { if root==nil{return 0};a,b:=maxDepth(root.Left),maxDepth(root.Right);if a>b{return a+1};return b+1 }
```

Python

```
def maxDepth(root):  
    if not root: return 0  
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

核心观察

对每个节点交换 left/right, 再递归处理两个子树。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量：空节点返回。随后通过“交换左右 → 递归左 → 递归右”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

不要把交换后的子树引用搞混；可前序也可后序。

1. 空节点返回



2. 交换左右

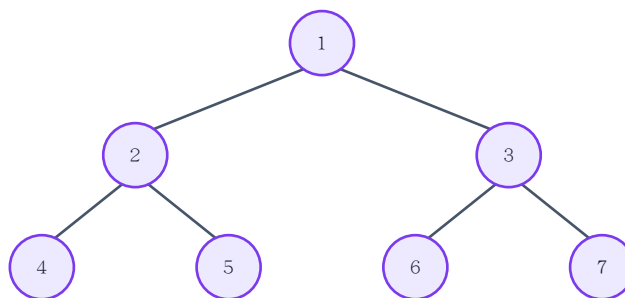


3. 递归左



4. 递归右

题目示意图



每个节点局部执行 $left \leftrightarrow right$, 整棵树得到镜像

扩展

对称树、树结构变换

Go

```
func invertTree(root *TreeNode) *TreeNode { if root==nil{return nil};root.Left,root.Right=invertTree(root.Right),invertTree(root.Left);return root }
```

Python

```
def invertTree(root):  
    if not root: return None  
    root.left, root.right = invertTree(root.right), invertTree(root.left)  
    return root
```

核心观察

同时比较一对镜像节点：值相等，且 $a.left$ 对 $b.right$ 、 $a.right$ 对 $b.left$ 。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量：比较根的左右子树。随后通过“值不同失败 → 外侧与外侧比较 → 内侧与内侧比较”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

不能比较同方向子树。

1. 比较根的左右子树



2. 值不同失败

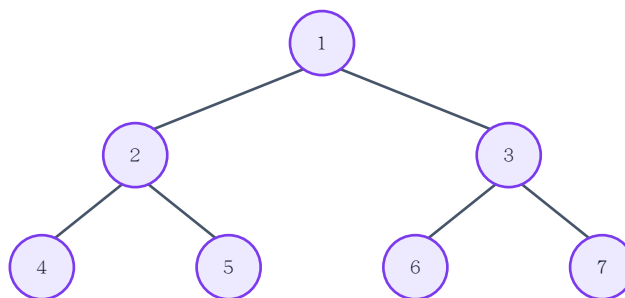


3. 外侧与外侧比较



4. 内侧与内侧比较

题目示意图



镜像递归： $L.left \square R.right$, $L.right \square R.left$

扩展

树同构、镜像判断

Go

```
func isSymmetric(root *TreeNode) bool { var f func(*TreeNode,*TreeNode)bool;f=func(a,b *TreeNode)bool{if a==nil||b==nil{return a==b};return a.Val==b.Val&&f(a.Left,b.Right)&&f(a.Right,b.Left)};return root==nil||f(root.Left,root.Right) }
```

Python

```
def isSymmetric(root):
    def same(a,b):
        if not a or not b: return a is b
        return a.val==b.val and same(a.left,b.right) and same(a.right,b.left)
    return same(root.left,root.right) if root else True
```

核心观察

后序 DFS 返回节点高度，同时用 `leftHeight+rightHeight` 更新经过当前节点的最长路径边数。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量：递归取左右高度。随后通过“更新 `ans=l+r` → 返回 `1+max(l,r)`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

LeetCode 直径定义为边数；返回高度和全局路径是两个不同量。

1. 递归取左右高度

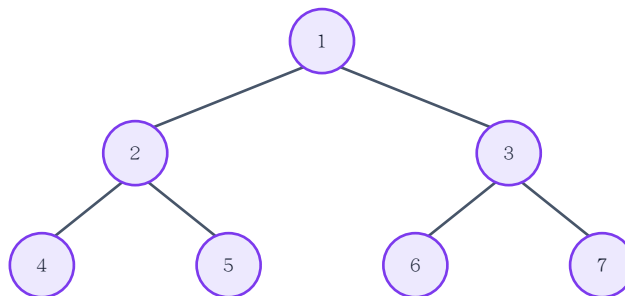


2. 更新 `ans=l+r`



3. 返回 `1+max(l,r)`

题目示意图



当前节点作为路径“拐点”，左右向下最长链长度相加就是经过它的直径

扩展

树 DP 路径问题、最大路径和

Go

```
func diameterOfBinaryTree(root *TreeNode) int { ans:=0;var dfs func(*TreeNode)int;dfs=func(n *TreeNode)int{if n==nil{return 0};l,r:=dfs(n.Left),dfs(n.Right);if l+r>ans{ans=l+r};if l>r{return l+1};return r+1};dfs(root);return ans }
```

Python

```
def diameterOfBinaryTree(root):  
    ans=0  
    def dfs(node):  
        nonlocal ans  
        if not node: return 0  
        l,r=dfs(node.left),dfs(node.right); ans=max(ans,l+r)  
        return 1+max(l,r)  
    dfs(root); return ans
```

核心观察

BFS 队列。每轮先固定 $size = \text{len}(\text{queue})$ ，只处理这 $size$ 个节点形成一层。

复杂度

$O(n)$ / $O(w)$

为什么这个算法成立

关键不变量：root 入队。随后通过“记录本层 $size \rightarrow$ 处理 $size$ 个节点 $\rightarrow$ 子节点入队并收集本层”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

层长度必须在处理本层前固定，不能随 push 动态变化。

1. root 入队



2. 记录本层 $size$

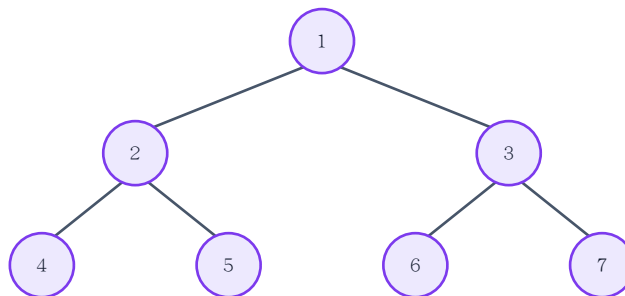


3. 处理 $size$ 个节点



4. 子节点入队并收集本层

题目示意图



queue: [level d] $\rightarrow$ 处理完后恰好只剩 [level d+1]

扩展

右视图、锯齿层序、最短路径 BFS

Go

```
func levelOrder(root *TreeNode) [][]int { if root==nil{return nil};q:=[]
*TreeNode{root};ans:=[][]int{};for len(q)>0{sz:=len(q);lv:=[]int{};for i
:=0;i<sz;i++{n:=q[0];q=q[1:];lv=append(lv,n.Val);if n.Left!=nil{q=append
(q,n.Left)};if n.Right!=nil{q=append(q,n.Right)}};ans=append(ans,lv)};re
turn ans }
```

Python

```
from collections import deque
def levelOrder(root):
    if not root: return []
    q=deque([root]); ans=[]
    while q:
        level=[]
        for _ in range(len(q)):
            n=q.popleft(); level.append(n.val)
            if n.left:q.append(n.left)
            if n.right:q.append(n.right)
        ans.append(level)
    return ans
```

核心观察

每个区间取中点为根，递归构造左半和右半，天然近似平衡。

复杂度

$O(n)$ / $O(\log n)$

为什么这个算法成立

关键不变量：选 mid 。随后通过“ $nums[mid]$ 做 root $\rightarrow$ 递归左半 $\rightarrow$ 递归右半”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

统一使用闭区间或半开区间，防止死递归。

1. 选 mid



2. $nums[mid]$ 做 root

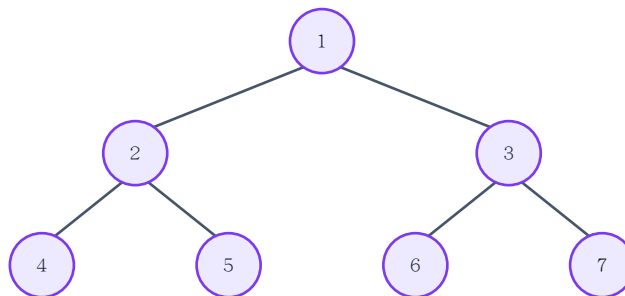


3. 递归左半



4. 递归右半

题目示意图



sorted $[1, 2, 3, 4, 5]$ 取 3 为根，两侧长度尽量平衡

扩展

有序链表转 BST、平衡 BST 构建

Go

```
func sortedArrayToBST(nums []int) *TreeNode {
    var f func(int,int)*TreeNode
    f = func(l,r int)*TreeNode{
        if l>=r{return nil};
        m:=(l+r)/2;
        n:=&TreeNode{Val:nums[m]};
        n.Left=f(l,m);
        n.Right=f(m+1,r);
        return n;
    }
    return f(0,len(nums))
}
```

Python

```
def sortedArrayToBST(nums):
    def build(l,r):
        if l>=r: return None
        m=(l+r)//2; root=TreeNode(nums[m])
        root.left=build(l,m); root.right=build(m+1,r)
        return root
    return build(0,len(nums))
```

核心观察

DFS 给每个节点传递合法开区间 $(low, high)$ ；左子树上界是 $root.val$ ，右子树下界是 $root.val$ 。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量：root 检查 $low < val < high$ 。随后通过“左递归 $(low, val) \rightarrow$ 右递归 $(val, high) \rightarrow$ 任一失败返回 false”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

只比较父子不够；BST 要求所有后代都满足上下界，且重复值不合法。

1. root 检查 $low < val < high$



2. 左递归 (low, val)

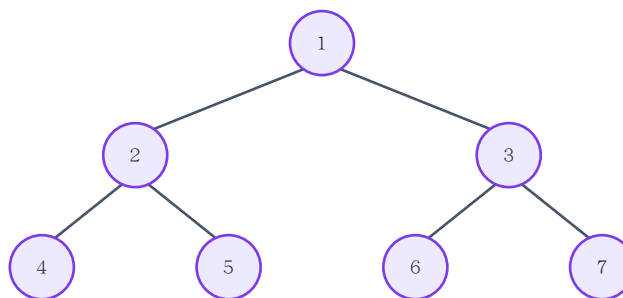


3. 右递归 $(val, high)$



4. 任一失败返回 false

题目示意图



节点合法范围从祖先一路收紧，而不是只看直接父节点

扩展

中序严格递增、BST 操作

Go

```
func isValidBST(root *TreeNode) bool { var f func(*TreeNode,int64,int64) bool;f=func(n *TreeNode,lo,hi int64)bool{if n==nil{return true};v:=int64(n.Val);return lo<v&&v<hi&&f(n.Left,lo,v)&&f(n.Right,v,hi)};return f(root,math.MinInt64,math.MaxInt64) }
```

Python

```
def isValidBST(root):
    def dfs(n,lo,hi):
        if not n: return True
        if not (lo<n.val<hi): return False
        return dfs(n.left,lo,n.val) and dfs(n.right,n.val,hi)
    return dfs(root,float('-inf'),float('inf'))
```

核心观察

BST 中序遍历递增；迭代中序每 pop 一个节点就 $k--$ ， $k==0$ 时返回。

复杂度

$O(h+k)$ / $O(h)$

为什么这个算法成立

关键不变量：沿左链入栈。随后通过“pop 最小尚未访问节点 $\rightarrow k-- \rightarrow k==0$ 返回”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

k 是 1-based；若有频繁查询可维护 subtree size。

1. 沿左链入栈



2. pop 最小尚未访问节点

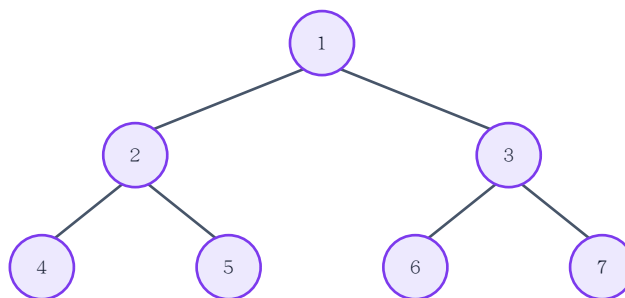


3. $k--$



4. $k==0$ 返回

题目示意图



中序序列就是 BST 的有序序列，因此第 k 个访问节点即答案

扩展

Order Statistic Tree、Kth largest

Go

```
func kthSmallest(root *TreeNode, k int) int { st:=[]*TreeNode{};cur:=root
;for{for cur!=nil{st=append(st,cur);cur=cur.Left};cur=st[len(st)-1];st=s
t[:len(st)-1];k--;if k==0{return cur.Val};cur=cur.Right} }
```

Python

```
def kthSmallest(root,k):
    st=[]; cur=root
    while True:
        while cur: st.append(cur); cur=cur.left
        cur=st.pop(); k-=1
        if k==0:return cur.val
        cur=cur.right
```

核心观察

BFS 每层取最后一个节点；或 DFS 先右后左，并在首次到达某深度时记录。

复杂度

$O(n)$ / $O(w)$ or $O(h)$

为什么这个算法成立

关键不变量：BFS 按层。随后通过“处理本层 $size \rightarrow i==size-1$ 时记录 $\rightarrow$ 继续下一层”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

DFS 版本关键是 $len(ans)==depth$ 才代表首次看到该层。

1. BFS 按层



2. 处理本层 $size$

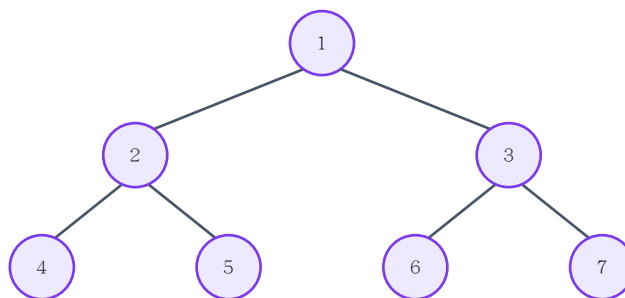


3. $i==size-1$ 时记录



4. 继续下一层

题目示意图



从右侧看，每一层仅能看到最右边的节点

扩展
左视图、边界视图

Go

```
func rightSideView(root *TreeNode) []int { if root==nil{return nil};q:=[]*TreeNode{root};ans:=[]int{};for len(q)>0{sz:=len(q);var last int;for i:=0;i<sz;i++{n:=q[0];q=q[1:];last=n.Val;if n.Left!=nil{q=append(q,n.Left)}};if n.Right!=nil{q=append(q,n.Right)}};ans=append(ans,last));return ans }
```

Python

```
from collections import deque
def rightSideView(root):
    if not root: return []
    q=deque([root]); ans=[]
    while q:
        for i in range(len(q)):
            n=q.popleft()
            if n.left:q.append(n.left)
            if n.right:q.append(n.right)
            if i==0: last=n.val
            else: last=n.val
        ans.append(last)
    return ans
```

核心观察

后序/逆前序维护 prev: 先处理 right, 再 left, 最后令 `root.right=prev`, `root.left=nil`。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量: 递归 right。随后通过“递归 left $\rightarrow$ `root.right=prev` $\rightarrow$ `root.left=nil`; `prev=root`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

最终所有 left 必须为 nil; 处理顺序决定是否丢失原右子树。

1. 递归 right



2. 递归 left

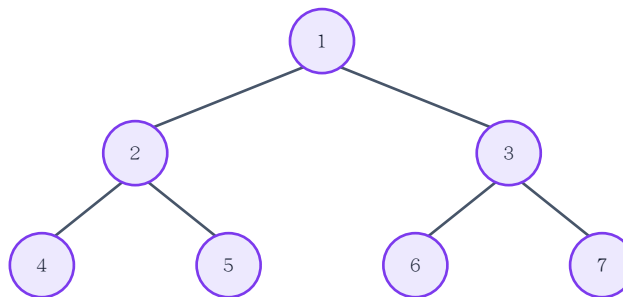


3. `root.right=prev`



4. `root.left=nil`; `prev=root`

题目示意图



按 root-left-right 的前序顺序展开为仅 right 指针的链表

扩展

原地树变链、Morris 风格 $O(1)$

Go

```
func flatten(root *TreeNode) { var prev *TreeNode; var f func(*TreeNode);  
f = func(n *TreeNode) { if n == nil { return }; f(n.Right); f(n.Left); n.Right = prev;  
n.Left = nil; prev = n }; f(root) }
```

Python

```
def flatten(root):  
    prev = None  
    def dfs(n):  
        nonlocal prev  
        if not n: return  
        dfs(n.right); dfs(n.left)  
        n.right = prev; n.left = None; prev = n  
    dfs(root)
```

#105 从前序与中序遍历序列构造二叉树 / Construct Binary Tree from Preorder and Inorder Traversal

核心观察

preorder 的首元素是根；用 inorder 的位置确定左子树大小，然后递归切分。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量：建 inorder 索引。随后通过“取 preorder 根 $\rightarrow$ 计算左子树长度 $\rightarrow$ 递归构造左右”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

先建 inorder value $\rightarrow$ index 哈希，否则重复扫描会 $O(n^2)$ ；注意区间边界。

1. 建 inorder 索引



2. 取 preorder 根

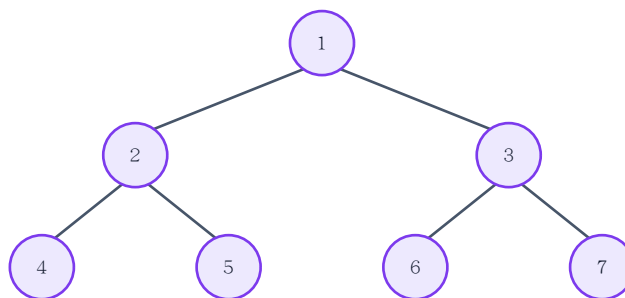


3. 计算左子树长度



4. 递归构造左右

题目示意图



pre: root | left | right; in: left | root | right

扩展

中序+后序重建、序列化/反序列化

Go

```
func buildTree(pre, in []int) *TreeNode { pos:=map[int]int{};for i,v:=range in{pos[v]=i};pi:=0;var f func(int,int)*TreeNode;f=func(l,r int)*TreeNode{if l>=r{return nil};v:=pre[pi];pi++;m:=pos[v];n:=&TreeNode{Val:v};n.Left=f(l,m);n.Right=f(m+1,r);return n};return f(0,len(in)) }
```

Python

```
def buildTree(preorder, inorder):  
    pos={v:i for i,v in enumerate(inorder)}; pi=0  
    def build(l,r):  
        nonlocal pi  
        if l>=r:return None  
        v=preorder[pi]; pi+=1; m=pos[v]; root=TreeNode(v)  
        root.left=build(l,m); root.right=build(m+1,r)  
        return root  
    return build(0,len(inorder))
```

核心观察

DFS 路径上维护 `prefixCount`。到节点当前和 `cur` 时，历史 `cur-target` 的次数就是以当前节点结束的合法路径数。

复杂度

$O(n)$ / $O(h)$ average map size

为什么这个算法成立

关键不变量: `cnt[0]=1`。随后通过“DFS 更新 `cur` $\rightarrow$ `ans += cnt[cur-target]` $\rightarrow$ 递归后回滚 `cnt[cur]`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

离开节点时必须 `prefixCount[cur]--`，否则会把他其他分支的前缀错误带入。

1. `cnt[0]=1`



2. DFS 更新 `cur`

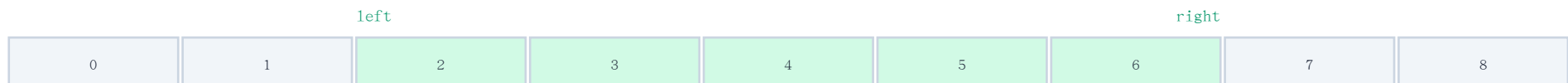


3. `ans += cnt[cur-target]`



4. 递归后回滚 `cnt[cur]`

题目示意图



root-to-current 路径上的 `prefix[j]-prefix[i]=target`，与数组前缀和完全同构

扩展

树上前缀和、子数组和 `k`

Go

```
func pathSum(root *TreeNode, target int) int { cnt:=map[int]int{0:1};ans:=0;var f func(*TreeNode,int);f=func(n *TreeNode,s int){if n==nil{return};s+=n.Val;ans+=cnt[s-target];cnt[s]++;f(n.Left,s);f(n.Right,s);cnt[s]--};f(root,0);return ans }
```

Python

```
def pathSum(root, target):
    cnt={0:1}; ans=0
    def dfs(n,s):
        nonlocal ans
        if not n:return
        s+=n.val; ans+=cnt.get(s-target,0); cnt[s]=cnt.get(s,0)+1
        dfs(n.left,s); dfs(n.right,s)
        cnt[s]-=1
    dfs(root,0); return ans
```

核心观察

若 $root$ 是 p/q 或 nil 直接返回；递归左右，若两边都非空，当前 $root$ 就是分叉点 LCA。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量：递归左右。随后通过“一边 nil 返回另一边 $\rightarrow$ 两边都非 nil 返回 $root \rightarrow$ 向上汇总”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

这是普通二叉树；BST 的 LCA 可以利用有序性更快决策。

1. 递归左右



2. 一边 nil 返回另一边

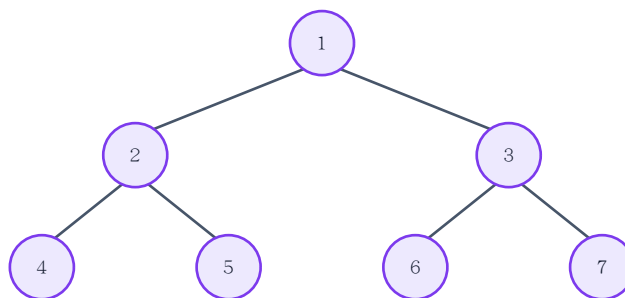


3. 两边都非 nil 返回 $root$



4. 向上汇总

题目示意图



p 与 q 若分居左右子树，当前节点是它们第一次汇合的祖先

扩展

多次 LCA 查询的倍增算法

Go

```
func lowestCommonAncestor(root,p,q *TreeNode) *TreeNode { if root==nil||  
root==p||root==q{return root};l:=lowestCommonAncestor(root.Left,p,q);r:=  
lowestCommonAncestor(root.Right,p,q);if l!=nil&&r!=nil{return root};if l  
!=nil{return l};return r }
```

Python

```
def lowestCommonAncestor(root,p,q):  
    if not root or root is p or root is q:return root  
    l=lowestCommonAncestor(root.left,p,q); r=lowestCommonAncestor(root.r  
ight,p,q)  
    if l and r:return root  
    return l or r
```

核心观察

后序返回“从当前节点向父节点延伸时能贡献的最大单支路径”；全局答案允许左右两支同时经过当前节点。

复杂度

$O(n)$ / $O(h)$

为什么这个算法成立

关键不变量: $left = \max(0, dfs(left))$ 。随后通过“ $right = \max(0, dfs(right)) \rightarrow ans = \max(ans, val + left + right) \rightarrow return\ val + \max(left, right)$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

全负数时 ans 必须初始化为负无穷；向上传递只能选左右一支。

1. $left = \max(0, dfs(left))$



2. $right = \max(0, dfs(right))$

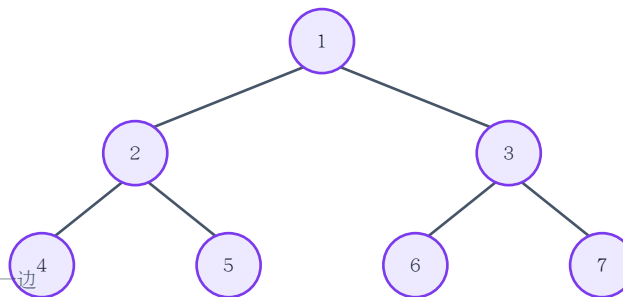


3. $ans = \max(ans, val + left + right)$



4. $return\ val + \max(left, right)$

题目示意图



全局路径可 $left \rightarrow root \rightarrow right$ ；但返回给父节点时不能形成叉，所以只能取一边。

扩展

树 DP 路径母题、直径加权版

Go

```
func maxPathSum(root *TreeNode) int { ans:=math.MinInt;var f func(*TreeNode)int;f=func(n *TreeNode)int{if n==nil{return 0};l,r:=f(n.Left),f(n.Right);if l<0{l=0};if r<0{r=0};if n.Val+l+r>ans{ans=n.Val+l+r};if l>r{return n.Val+l};return n.Val+r};f(root);return ans }
```

Python

```
def maxPathSum(root):  
    ans=float('-inf')  
    def dfs(n):  
        nonlocal ans  
        if not n:return 0  
        l=max(0,dfs(n.left)); r=max(0,dfs(n.right))  
        ans=max(ans,n.val+l+r)  
        return n.val+max(l,r)  
    dfs(root); return ans
```

核心观察

遍历网格，遇到未访问陆地就启动一次 DFS/BFS flood fill，把整块岛屿全部标记；启动次数即岛屿数。

复杂度

$O(mn)$ / $O(mn)$ worst recursion/queue

为什么这个算法成立

关键不变量：扫描所有格子。随后通过“遇 1 计数+1 → DFS/BFS 四方向淹没 → 继续扫描”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

可直接把 grid 中的 1 改成 0 省 visited；注意四方向边界。

1. 扫描所有格子



2. 遇 1 计数+1

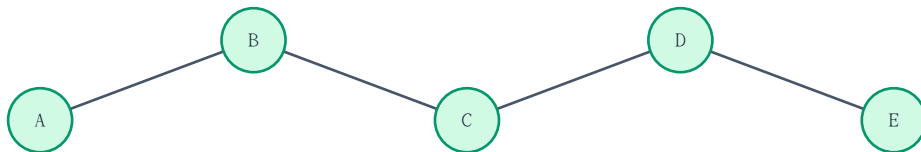


3. DFS/BFS 四方向淹没



4. 继续扫描

题目示意图



每次 flood fill 会把一个连通分量完整消除，因此计数的是连通分量数

扩展

岛屿面积、周长、并查集

Go

```
func numIslands(g [][]byte) int { m,n:=len(g),len(g[0]);var dfs func(int,int);dfs=func(r,c int){if r<0||r>=m||c<0||c>=n||g[r][c]!='1'{return};g[r][c]='0';dfs(r+1,c);dfs(r-1,c);dfs(r,c+1);dfs(r,c-1)};ans:=0;for r:=0;r<m;r++){for c:=0;c<n;c++){if g[r][c]=='1'{ans++;dfs(r,c)}}};return ans }
```

Python

```
def numIslands(grid):
    m,n=len(grid),len(grid[0])
    def dfs(r,c):
        if r<0 or r>=m or c<0 or c>=n or grid[r][c]!='1':return
        grid[r][c]='0'
        dfs(r+1,c);dfs(r-1,c);dfs(r,c+1);dfs(r,c-1)
    ans=0
    for r in range(m):
        for c in range(n):
            if grid[r][c]=='1': ans+=1; dfs(r,c)
    return ans
```

核心观察

把所有初始腐烂橘子作为多源同时入队，逐层 BFS 扩散；每一层代表 1 分钟。

复杂度

$O(mn)$ / $O(mn)$

为什么这个算法成立

关键不变量：统计 fresh。随后通过“所有 rotten 入队 → 分层扩散并 fresh-- → fresh==0 返回分钟数否则 -1”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

时间只在确实扩散一层时增加；最后要检查 fresh 是否为 0。

1. 统计 fresh



2. 所有 rotten 入队

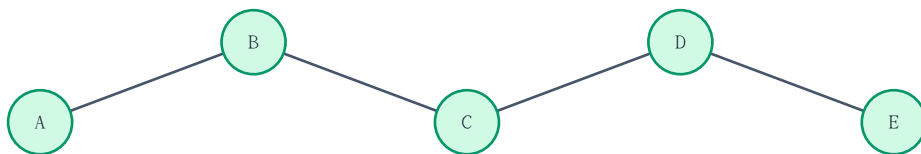


3. 分层扩散并 fresh--



4. fresh==0 返回分钟数否则 -1

题目示意图



多个腐烂源同时向外扩张，BFS 层数就是最早到达时间

扩展

多源最短路、01 矩阵、感染传播

Go

```
func orangesRotting(g [][]int) int { m,n:=len(g),len(g[0]);q:=[][2]int{};fresh:=0;for r:=0;r<m;r++{for c:=0;c<n;c++{if g[r][c]==2{q=append(q,[2]int{r,c})}else if g[r][c]==1{fresh++}}};mins:=0;dirs:=[][2]int{{1,0},{-1,0},{0,1},{0,-1}};for len(q)>0&&fresh>0{sz:=len(q);for i:=0;i<sz;i++{p:=q[0];q=q[1:];for _,d:=range dirs{r,c:=p[0]+d[0],p[1]+d[1];if r>=0&&r<m&&c>=0&&c<n&&g[r][c]==1{g[r][c]=2;fresh--;q=append(q,[2]int{r,c})}}};mins++};if fresh>0{return -1};return mins }
```

Python

```
from collections import deque
def orangesRotting(grid):
    m,n=len(grid),len(grid[0]); q=deque(); fresh=0
    for r in range(m):
        for c in range(n):
            if grid[r][c]==2:q.append((r,c))
            elif grid[r][c]==1:fresh+=1
    minutes=0
    while q and fresh:
        for _ in range(len(q)):
            r,c=q.popleft()
            for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)):
                nr,nc=r+dr,c+dc
                if 0<=nr<m and 0<=nc<n and grid[nr][nc]==1:
                    grid[nr][nc]=2; fresh-=1; q.append((nr,nc))
        minutes+=1
    return minutes if fresh==0 else -1
```

核心观察

构建 `prereq -> course` 的有向边和 `indegree`; Kahn BFS 不断取入度为 0 的课程。处理数==numCourses 表示无环。

复杂度

$O(V+E)$ / $O(V+E)$

为什么这个算法成立

关键不变量：建图和入度。随后通过“所有 `indegree=0` 入队 $\rightarrow$ pop 后降低邻居入度 $\rightarrow$ 统计处理节点数”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

边方向很容易写反；有环时队列最终会提前耗尽。

1. 建图和入度



2. 所有 `indegree=0` 入队

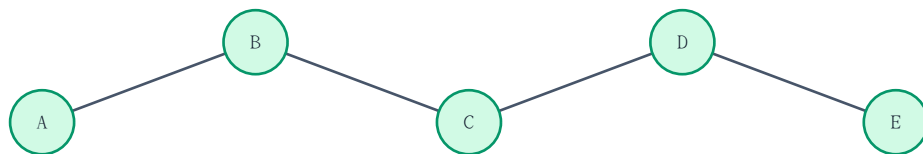


3. pop 后降低邻居入度



4. 统计处理节点数

题目示意图



依赖图中“可立即学习”的课程就是入度 0 节点

扩展

输出拓扑序、构建系统依赖、DAG DP

Go

```
func canFinish(n int, pre [][]int) bool { g:=make([][]int,n);deg:=make([]int,n);for _,e:=range pre{g[e[1]]=append(g[e[1]],e[0]);deg[e[0]]++;q:=[]int{};for i,d:=range deg{if d==0{q=append(q,i)}};seen:=0;for len(q)>0{u:=q[0];q=q[1:];seen++;for _,v:=range g[u]{deg[v]--;if deg[v]==0{q=append(q,v)}}};return seen==n }
```

Python

```
from collections import deque
def canFinish(numCourses, prerequisites):
    g=[[] for _ in range(numCourses)]; indeg=[0]*numCourses
    for a,b in prerequisites:g[b].append(a);indeg[a]+=1
    q=deque(i for i,d in enumerate(indeg) if d==0); seen=0
    while q:
        u=q.popleft(); seen+=1
        for v in g[u]:
            indeg[v]-=1
            if indeg[v]==0:q.append(v)
    return seen==numCourses
```

核心观察

每个节点维护 children 与 isEnd。insert 沿字符创建节点；search 必须最终 isEnd；startsWith 只要求路径存在。

复杂度

$O(L)$ per op / $O(\text{total chars})$

为什么这个算法成立

关键不变量：从 root 开始。随后通过“逐字符沿 children $\rightarrow$ 缺少则按操作创建/失败 $\rightarrow$ 终点处理 isEnd”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

search 与 startsWith 的区别在终点是否要求 isEnd。

1. 从 root 开始



2. 逐字符沿 children

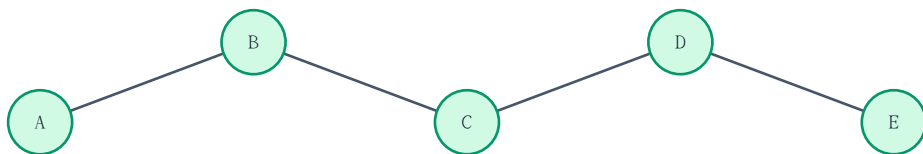


3. 缺少则按操作创建/失败



4. 终点处理 isEnd

题目示意图



cat 与 car 共享 c-a 前缀节点，第三层才分叉

扩展

前缀计数、自动补全、AC 自动机

Go

```
type Trie struct{Ch [26]*Trie;End bool}
func Constructor() Trie{return Trie{}}
func(t *Trie)Insert(w string){n:=t;for _,c:=range []byte(w){i:=c-'a';if
n.Ch[i]==nil{n.Ch[i]=&Trie{}};n=n.Ch[i];n.End=true}
func(t *Trie)walk(s string)*Trie{n:=t;for _,c:=range []byte(s){i:=c-'a';
if n.Ch[i]==nil{return nil};n=n.Ch[i];return n}
func(t *Trie)Search(w string)bool{n:=t.walk(w);return n!=nil&& n.End}
func(t *Trie)StartsWith(p string)bool{return t.walk(p)!=nil}
```

Python

```
class Trie:
    def __init__(self): self.ch={}; self.end=False
    def insert(self,word):
        n=self
        for c in word:n=n.ch.setdefault(c,Trie())
        n.end=True
    def _walk(self,s):
        n=self
        for c in s:
            if c not in n.ch:return None
            n=n.ch[c]
        return n
    def search(self,word):
        n=self._walk(word); return bool(n and n.end)
    def startsWith(self,prefix): return self._walk(prefix) is not None
```

核心观察

path 表示当前排列, used 标记哪些元素已选; 每层从所有未使用元素中选择一个。

复杂度

$O(n \cdot n!)$ / $O(n)$ recursion excluding output

为什么这个算法成立

关键不变量: 若 $\text{len}(\text{path}) == n$ 收集。随后通过“枚举所有 $i \rightarrow$ 跳过 $\text{used}[i] \rightarrow$ 选择 $\rightarrow$ 递归 $\rightarrow$ 撤销”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

保存答案时必须复制 path, 而不是保存同一可变切片/列表引用。

1. 若 $\text{len}(\text{path}) == n$ 收集



2. 枚举所有 i

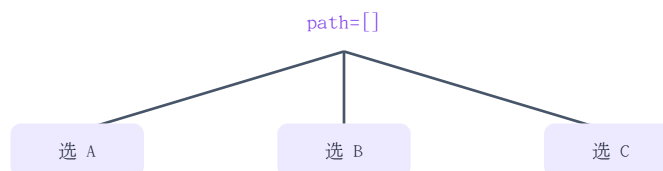


3. 跳过 $\text{used}[i]$



4. 选择 $\rightarrow$ 递归 $\rightarrow$ 撤销

题目示意图



递归树每一层选择剩余未使用元素, 叶子节点是一种排列

扩展

含重复元素排列、排列剪枝

Go

```
func permute(nums []int) [][]int { ans:=[][]int{};path:=[]int{};used:=make([]bool,len(nums));var f func();f=func(){if len(path)==len(nums){cp:=append([]int(nil),path...);ans=append(ans,cp);return};for i,x:=range nums{if used[i]{continue};used[i]=true;path=append(path,x);f();path=path[:len(path)-1];used[i]=false}};f();return ans }
```

Python

```
def permute(nums):
    ans=[]; path=[]; used=[False]*len(nums)
    def dfs():
        if len(path)==len(nums): ans.append(path.copy()); return
        for i,x in enumerate(nums):
            if used[i]:continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    se
    dfs(); return ans
```

核心观察

每个递归节点本身就是一个合法子集；用 `start` 限制下一次只能选后面的元素，避免重复。

复杂度

$O(n \cdot 2^n)$ / $O(n)$

为什么这个算法成立

关键不变量：进入 `dfs` 先保存 `path`。随后通过 “for `i` from `start` → 选择 `nums[i]` → `dfs(i+1)` 后撤销” 不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

答案收集发生在每个节点，而不仅是叶子。

1. 进入 `dfs` 先保存 `path`



2. for `i` from `start`

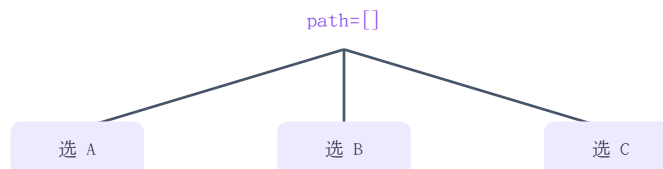


3. 选择 `nums[i]`



4. `dfs(i+1)` 后撤销

题目示意图



决策树上的所有节点，而不仅叶子，都对应一个子集

扩展

Subsets II、组合枚举

Go

```
func subsets(nums []int) [][]int { ans:=[][]int{};path:=[]int{};var f func(int);f=func(start int){ans=append(ans,append([]int(nil),path...));for i:=start;i<len(nums);i++){path=append(path,nums[i]);f(i+1);path=path[:len(path)-1]};f(0);return ans }
```

Python

```
def subsets(nums):  
    ans=[]; path=[]  
    def dfs(start):  
        ans.append(path.copy())  
        for i in range(start,len(nums)):  
            path.append(nums[i]); dfs(i+1); path.pop()  
    dfs(0); return ans
```

核心观察

digits 的每一位对应一个字母集合；递归深度等于 digit 下标，枚举该集合中的每个字符。

复杂度

$O(4^n)$ / $O(n)$

为什么这个算法成立

关键不变量：pos 指向当前 digit。随后通过“枚举映射字母 → append 字符并递归 pos+1 → 到末尾收集”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

空输入返回空结果；2-9 的映射要准确。

1. pos 指向当前 digit



2. 枚举映射字母

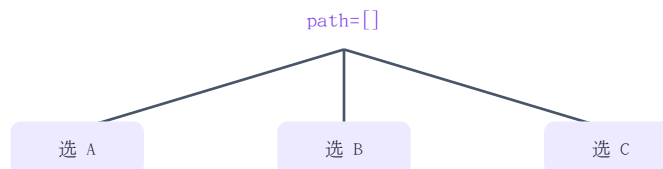


3. append 字符并递归 pos+1



4. 到末尾收集

题目示意图



23 -> {a,b,c} x {d,e,f}, 形成 9 个叶子

扩展

笛卡尔积、组合生成

Go

```
func letterCombinations(d string) []string { if len(d)==0{return nil};mp
:=[]string{"","","abc","def","ghi","jkl","mno","pqrs","tuv","wxyz"};ans:
=[]string{};path:=make([]byte,0,len(d));var f func(int);f=func(i int){if
i==len(d){ans=append(ans,string(path));return};for _,c:=range []byte(mp
[d[i]-'0']){path=append(path,c);f(i+1);path=path[:len(path)-1]}};f(0);re
turn ans }
```

Python

```
def letterCombinations(digits):
    if not digits: return []
    mp={'2':'abc','3':'def','4':'ghi','5':'jkl','6':'mno','7':'pqrs','8'
:'tuv','9':'wxyz'}
    ans=[]; path=[]
    def dfs(i):
        if i==len(digits): ans.append(''.join(path)); return
        for c in mp[digits[i]]: path.append(c); dfs(i+1); path.pop()
    dfs(0); return ans
```

核心观察

排序后从 `start` 开始枚举；因为元素可重复使用，递归时仍传 `i`；`target < 0` 或 `candidate > target` 时剪枝。

复杂度

指数级 / $O(2^{\text{target}/\text{minVal}})$ recursion

为什么这个算法成立

关键不变量：排序可剪枝。随后通过“枚举 $i \geq \text{start} \rightarrow$ 选择 `candidate` $\rightarrow$ 递归 `remain-candidate`，撤销”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

“可重复”意味着 `dfs(i, remain-candidate)`，不是 `i+1`。

1. 排序可剪枝



2. 枚举 $i \geq \text{start}$

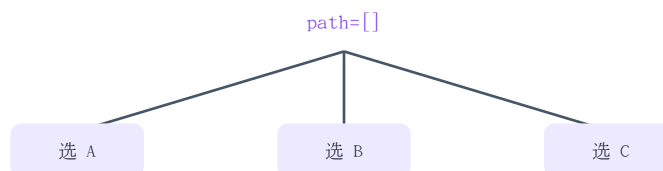


3. 选择 `candidate`



4. 递归 `remain-candidate`，撤销

题目示意图



`candidate 2` 被选后下一层仍允许选 `2`，因此路径可出现 `[2,2,3]`

扩展

Combination Sum II/III、背包枚举

Go

```
func combinationSum(c []int, target int) [][]int { sort.Ints(c); ans:=[][]int{}; path:=[]int{}; var f func(int,int); f=func(start,remain int){if remain==0{ans=append(ans,append([]int(nil),path...));return};for i:=start;i<len(c)&&c[i]<=remain;i++){path=append(path,c[i]);f(i,remain-c[i]);path=path[:len(path)-1]}};f(0,target);return ans }
```

Python

```
def combinationSum(candidates, target):
    candidates.sort(); ans=[]; path=[]
    def dfs(start, remain):
        if remain==0: ans.append(path.copy()); return
        for i in range(start, len(candidates)):
            x=candidates[i]
            if x>remain:break
            path.append(x); dfs(i, remain-x); path.pop()
    dfs(0, target); return ans
```

核心观察

只在 $\text{open} < n$ 时放左括号，只在 $\text{close} < \text{open}$ 时放右括号，从构造过程就保证前缀合法。

复杂度

Catalan 规模 / $O(n)$

为什么这个算法成立

关键不变量：状态 $(\text{open}, \text{close})$ 。随后通过“ $\text{open} < n$ 可加 ($\rightarrow$ $\text{close} < \text{open}$ 可加) $\rightarrow$ 长度 $2n$ 收集”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

不要先生成所有 $2n$ 长字符串再检查，剪枝应发生在构造时。

1. 状态 $(\text{open}, \text{close})$



2. $\text{open} < n$ 可加 (

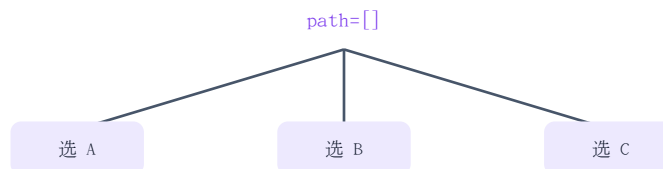


3. $\text{close} < \text{open}$ 可加)



4. 长度 $2n$ 收集

题目示意图



任何前缀都必须满足 $\text{close} \leq \text{open}$ ，叶子自动合法

扩展

合法括号、Catalan 数

Go

```
func generateParenthesis(n int) []string { ans:=[]string{};path:=[]byte{};var f func(int,int);f=func(op,cl int){if len(path)==2*n{ans=append(ans,string(path));return};if op<n{path=append(path,'(');f(op+1,cl);path=path[:len(path)-1]};if cl<op{path=append(path,')');f(op,cl+1);path=path[:len(path)-1]}};f(0,0);return ans }
```

Python

```
def generateParenthesis(n):  
    ans=[]; path=[]  
    def dfs(op,cl):  
        if len(path)==2*n: ans.append(''.join(path)); return  
        if op<n: path.append('(');dfs(op+1,cl);path.pop()  
        if cl<op: path.append(')');dfs(op,cl+1);path.pop()  
    dfs(0,0);return ans
```

核心观察

从每个格子尝试 DFS；匹配当前字符后临时标记 `visited`，向四方向寻找下一个字符，返回前恢复。

复杂度

$O(mn \cdot 4^L)$ / $O(L)$

为什么这个算法成立

关键不变量：枚举起点。随后通过“字符不匹配立即失败 → 标记当前格 → 四方向递归后恢复”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

同一条路径不能重复使用格子；成功路径可提前返回减少回溯。

1. 枚举起点



2. 字符不匹配立即失败

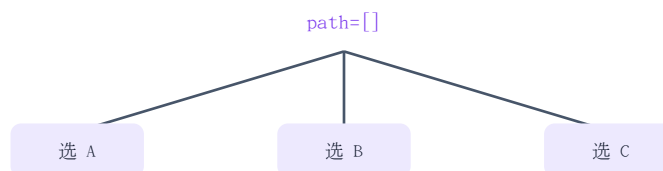


3. 标记当前格



4. 四方向递归后恢复

题目示意图



路径像在网格上走迷宫，每个格子在当前路径中最多使用一次

扩展

Trie + 多单词搜索、网格 DFS

Go

```
func exist(b [][]byte, w string) bool { m, n := len(b), len(b[0]); var f func(int, int, int) bool; f = func(r, c, i int) bool { if i == len(w) { return true }; if r < 0 || r >= m || c < 0 || c >= n || b[r][c] != w[i] { return false }; ch := b[r][c]; b[r][c] = '#'; ok := f(r+1, c, i+1) || f(r-1, c, i+1) || f(r, c+1, i+1) || f(r, c-1, i+1); b[r][c] = ch; return ok }; for r := 0; r < m; r++ { for c := 0; c < n; c++ { if f(r, c, 0) { return true } } }; return false }
```

Python

```
def exist(board, word):
    m, n = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != word[i]: return False
        ch = board[r][c]; board[r][c] = '#'
        ok = dfs(r+1, c, i+1) or dfs(r-1, c, i+1) or dfs(r, c+1, i+1) or dfs(r, c-1, i+1)
        board[r][c] = ch; return ok
    return any(dfs(r, c, 0) for r in range(m) for c in range(n))
```

核心观察

`start` 表示下一个待切位置，枚举 `end`；只有 `s[start:end]` 是回文时才把这一段加入 `path` 继续递归。

复杂度

$O(n \cdot 2^n)$ / $O(n)$

为什么这个算法成立

关键不变量：枚举 `end`。随后通过“判断当前段是否回文 → 加入 `path` → `dfs(end)` 后撤销”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

切片边界与回文判断；可预处理 `palindrome DP` 降低重复判断。

1. 枚举 `end`



2. 判断当前段是否回文

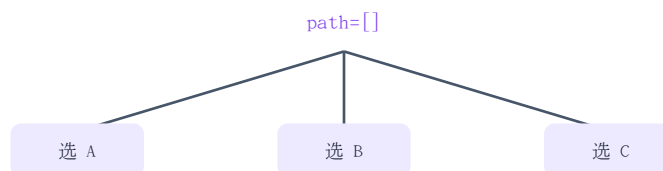


3. 加入 `path`



4. `dfs(end)` 后撤销

题目示意图



aab: 先切 `a|a|b`，也可切 `aa|b`；只有回文段才能成为边

扩展

回文 DP + 回溯、最少切割

Go

```
func partition(s string) [][]string { ans:=[][]string{};path:=[]string{}  
;pal:=func(a,b int)bool{for a<b{if s[a]!=s[b]{return false};a++;b--};ret  
urn true};var f func(int);f=func(st int){if st==len(s){ans=append(ans,ap  
pend([]string(nil),path...)};return};for e:=st;e<len(s);e++){if pal(st,e)  
{path=append(path,s[st:e+1]);f(e+1);path=path[:len(path)-1]}}};f(0);retu  
rn ans }
```

Python

```
def partition(s):  
    ans=[]; path=[]  
    def pal(a,b):  
        while a<b:  
            if s[a]!=s[b]:return False  
            a+=1;b-=1  
        return True  
    def dfs(start):  
        if start==len(s):ans.append(path.copy());return  
        for end in range(start,len(s)):  
            if pal(start,end): path.append(s[start:end+1]);dfs(end+1);pa  
th.pop()  
        dfs(0);return ans
```

核心观察

逐行放皇后；用 `cols`、`diag1=r-c`、`diag2=r+c` 三个集合 $O(1)$ 判断冲突。

复杂度

$O(n!)$ / $O(n)$

为什么这个算法成立

关键不变量：row 从 0 到 $n-1$ 。随后通过“枚举 col → 检查三类冲突 → 放置->下一行->撤销”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

对角线编码要一致；每层恰好放一个皇后，因此无需 `rows` 集合。

1. row 从 0 到 $n-1$



2. 枚举 col

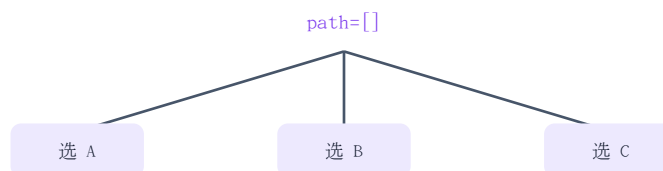


3. 检查三类冲突



4. 放置->下一行->撤销

题目示意图



同列冲突由 `c` 相同；主对角线 `r-c` 相同；副对角线 `r+c` 相同

扩展

位运算 N 皇后、数独求解

Go

```
func solveNQueens(n int) [][]string { ans:=[][]string{};cols:=map[int]bool{};d1:=map[int]bool{};d2:=map[int]bool{};pos:=make([]int,n);var f func(int);f=func(r int){if r==n{board:=make([]string,n);for i,c:=range pos{row:=make([]byte,n);for j:=range row{row[j]='.'};row[c]='Q';board[i]=string(row)};ans=append(ans,board);return};for c:=0;c<n;c++{if cols[c]||d1[r-c]||d2[r+c]{continue};cols[c]=true;d1[r-c]=true;d2[r+c]=true;pos[r]=c;f(r+1);delete(cols,c);delete(d1,r-c);delete(d2,r+c)}};f(0);return ans }
```

Python

```
def solveNQueens(n):
    ans=[]; cols=set(); d1=set(); d2=set(); board=[['.']*n for _ in range(n)]
    def dfs(r):
        if r==n: ans.append(''.join(row) for row in board); return
        for c in range(n):
            if c in cols or r-c in d1 or r+c in d2:continue
            cols.add(c);d1.add(r-c);d2.add(r+c);board[r][c]='Q';dfs(r+1)
            board[r][c]='.';cols.remove(c);d1.remove(r-c);d2.remove(r+c)
    dfs(0);return ans
```

核心观察

标准 lower_bound: 在半开区间 $[1, r)$ 中找第一个 $\geq \text{target}$ 的位置。

复杂度

$O(\log n)$ / $O(1)$

为什么这个算法成立

关键不变量: $l=0, r=n$ 。随后通过 “ $m=(l+r)//2 \rightarrow \text{nums}[m] < \text{target} \Rightarrow l=m+1 \rightarrow$ 否则 $r=m$ ” 不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

统一模板后不要混用闭区间边界。

1. $l=0, r=n$



2. $m=(l+r)//2$



3. $\text{nums}[m] < \text{target} \Rightarrow l=m+1$



4. 否则 $r=m$

题目示意图



循环结束 $l=r$, 且左边都 $< \text{target}$, 当前位置是最左可插入点

扩展

所有“第一个满足条件”的边界二分

Go

```
func searchInsert(nums []int, target int) int { l,r:=0,len(nums);for l<r{
m:=(l+r)/2;if nums[m]<target{l=m+1}else{r=m}};return l }
```

Python

```
def searchInsert(nums, target):
    l, r = 0, len(nums)
    while l < r:
        m = (l + r) // 2
        if nums[m] < target: l = m + 1
        else: r = m
    return l
```

核心观察

矩阵每行有序且下一行首元素大于上一行尾元素，可把 $m \times n$ 矩阵看成一维有序数组。

复杂度

$O(\log(mn)) / O(1)$

为什么这个算法成立

关键不变量: $l=0, r=m \times n$ 。随后通过“mid 映射到 $\text{matrix}[\text{mid}/n][\text{mid}\%n] \rightarrow$ 比较 $\text{target} \rightarrow$ 标准二分”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

一维 index 转二维: $\text{row}=\text{idx}/n, \text{col}=\text{idx}\%n$ 。

1. $l=0, r=m \times n$



2. mid 映射到 $\text{matrix}[\text{mid}/n][\text{mid}\%n]$



3. 比较 target



4. 标准二分

题目示意图



不用真的 flatten; 只把索引空间 flatten 即可

扩展
虚拟数组、二分映射

Go

```
func searchMatrix(a [][]int, target int) bool { m,n:=len(a),len(a[0]);l,r:=0,m*n;for l<r{mid:=(l+r)/2;x:=a[mid/n][mid%n];if x<target{l=mid+1}else{r=mid}};return l<m*n&& a[l/n][l%n]==target }
```

Python

```
def searchMatrix(matrix, target):  
    m,n=len(matrix),len(matrix[0]); l,r=0,m*n  
    while l<r:  
        mid=(l+r)//2; x=matrix[mid//n][mid%n]  
        if x<target:l=mid+1  
        else:r=mid  
    return l<m*n and matrix[l//n][l%n]==target
```

#34 在排序数组中查找元素的第一个和最后一个位置 / Find First and Last Position of Element in Sorted Array

核心观察

两次边界二分: `left=lower_bound(target)`, `right=lower_bound(target+1)-1` (整数场景) 或独立 `upper_bound`。

复杂度

$O(\log n)$ / $O(1)$

为什么这个算法成立

关键不变量: 求左边界。随后通过“验证 `target` 存在 $\rightarrow$ 求右开边界 $\rightarrow$ 返回 `[left, right-1]`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

先检查 `left < n` $\&\&$ `nums[left] == target`, 否则不存在。

1. 求左边界



2. 验证 `target` 存在



3. 求右开边界



4. 返回 `[left, right-1]`

题目示意图



目标区间本质是 `[first >= target, first > target)`

扩展

`lower_bound` / `upper_bound` 通用模板

Go

```
func searchRange(nums []int, target int) []int { lb:=func(x int)int{l,r:=0,len(nums);for l<r{m:=(l+r)/2;if nums[m]<x{l=m+1}else{r=m}};return l};a:=lb(target);if a==len(nums)||nums[a]!=target{return []int{-1,-1}};return []int{a,lb(target+1)-1} }
```

Python

```
def searchRange(nums, target):  
    def lb(x):  
        l, r = 0, len(nums)  
        while l < r:  
            m = (l + r) // 2  
            if nums[m] < x: l = m + 1  
            else: r = m  
        return l  
    a = lb(target)  
    if a == len(nums) or nums[a] != target: return [-1, -1]  
    return [a, lb(target + 1) - 1]
```

核心观察

每轮至少有一半有序。判断 `target` 是否落在有序半区，决定丢弃哪一半。

复杂度

$O(\log n)$ / $O(1)$

为什么这个算法成立

关键不变量：计算 `mid`。随后通过“判断左半或右半有序 → 判断 `target` 是否在有序半 → 缩小区间”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

比较边界时注意闭区间；无重复元素是该判断成立的重要条件。

1. 计算 `mid`



2. 判断左半或右半有序



3. 判断 `target` 是否在有序半



4. 缩小区间

题目示意图



旋转数组虽然整体无序，但 `mid` 切开后至少一半保持严格有序

扩展

有重复元素旋转数组、找最小值

Go

```
func search(nums []int, target int) int { l, r := 0, len(nums)-1; for l <= r { m := (l+r)/2; if nums[m] == target { return m }; if nums[l] <= nums[m] { if nums[l] <= target && target < nums[m] { r = m-1 } else { l = m+1 } } else { if nums[m] < target && target <= nums[r] { l = m+1 } else { r = m-1 } } }; return -1 }
```

Python

```
def search(nums, target):  
    l, r = 0, len(nums)-1  
    while l <= r:  
        m = (l+r)//2  
        if nums[m] == target: return m  
        if nums[l] <= nums[m]:  
            if nums[l] <= target < nums[m]: r = m-1  
            else: l = m+1  
        else:  
            if nums[m] < target <= nums[r]: l = m+1  
            else: r = m-1  
    return -1
```

核心观察

比较 `nums[mid]` 与 `nums[right]`: 若 `mid` 更大, 最小值一定在 `mid` 右侧; 否则最小值在 `[left, mid]`。

复杂度

$O(\log n)$ / $O(1)$

为什么这个算法成立

关键不变量: `l=0, r=n-1`。随后通过 “`m → nums[m]>nums[r] => l=m+1`” 不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

右边界不能在 `else` 中写 `mid-1`, 因为 `mid` 可能就是最小值。

1. `l=0, r=n-1`



2. `m`



3. `nums[m]>nums[r] => l=m+1`



4. `else r=m`

题目示意图



右端值提供 “是否跨过旋转断点” 的参照

扩展

旋转数组查找、重复元素版本

Go

```
func findMin(nums []int) int { l,r:=0,len(nums)-1;for l<r{m:=(l+r)/2;if nums[m]>nums[r]{l=m+1}else{r=m}};return nums[l] }
```

Python

```
def findMin(nums):  
    l,r=0,len(nums)-1  
    while l<r:  
        m=(l+r)//2  
        if nums[m]>nums[r]:l=m+1  
        else:r=m  
    return nums[l]
```

核心观察

在较短数组上二分分割点 i ，另一数组分割点 j 由总左半长度确定；要求 $\maxLeft \leq \minRight$ 。

复杂度

$O(\log \min(m, n)) / O(1)$

为什么这个算法成立

关键不变量：保证 A 更短。随后通过“二分 i 并算 $j \rightarrow$ 检查 $A_{left} \leq B_{right}$ 且 $B_{left} \leq A_{right} \rightarrow$ 根据奇偶返回中位数”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

边界需要 $\pm infinity$ 哨兵；始终在较短数组上二分。

1. 保证 A 更短



2. 二分 i 并算 j



3. 检查 $A_{left} \leq B_{right}$ 且 $B_{left} \leq A_{right}$



4. 根据奇偶返回中位数

题目示意图



A 与 B 各切一刀，使左侧总元素数等于右侧，并满足左最大 $\leq$ 右最小

扩展

第 k 小、两个有序数组 partition 技巧

Go

```
func findMedianSortedArrays(a,b []int) float64 { if len(a)>len(b){a,b=b,a};m,n:=len(a),len(b);half:=(m+n+1)/2;l,r:=0,m;neg,pos:=math.Inf(-1),math.Inf(1);for l<=r{i:=(l+r)/2;j:=half-i;al,ar,bl,br:=neg,pos,neg,pos;if i>0{al=float64(a[i-1])};if i<m{ar=float64(a[i])};if j>0{bl=float64(b[j-1])};if j<n{br=float64(b[j])};if al<=br&&bl<=ar{if (m+n)%2==1{return math.Max(al,bl)};return (math.Max(al,bl)+math.Min(ar,br))/2};if al>br{r=i-1}else{l=i+1}};return 0 }
```

Python

```
def findMedianSortedArrays(A,B):
    if len(A)>len(B):A,B=B,A
    m,n=len(A),len(B); half=(m+n+1)//2; l,r=0,m
    while l<=r:
        i=(l+r)//2; j=half-i
        al=A[i-1] if i else float('-inf'); ar=A[i] if i<m else float('inf')
        bl=B[j-1] if j else float('-inf'); br=B[j] if j<n else float('inf')
        if al<=br and bl<=ar:
            if (m+n)%2:return max(al,bl)
            return (max(al,bl)+min(ar,br))/2
        if al>br:r=i-1
        else:l=i+1
```

核心观察

左括号入栈；遇右括号必须与栈顶对应，否则立即失败；结束时栈必须为空。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量：遍历字符。随后通过“左括号 push → 右括号检查并 pop → 最终 stack empty”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

空栈遇右括号要先判断；不要只统计括号数量。

1. 遍历字符



2. 左括号 push

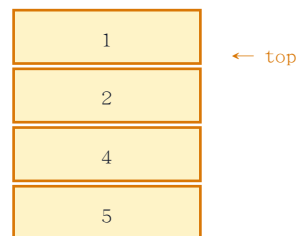


3. 右括号检查并 pop



4. 最终 stack empty

题目示意图



栈保存“尚未闭合”的左括号，后来的右括号只能闭合最近一个

扩展

表达式解析、嵌套结构

Go

```
func isValid(s string) bool { mp:=map[byte]byte{'(':')','[':']','{':'}':{'(':')','[':']','{':'}};st:=[]byte{};for i:=range s{c:=s[i];if op,ok:=mp[c];ok{if len(st)==0||s
t[len(st)-1]!=op{return false};st=st[:len(st)-1]}else{st=append(st,c)}};
return len(st)==0 }
```

Python

```
def isValid(s):
    mp={'(':')','[':']','{':'}':{'(':')','[':']','{':'}}; st=[]
    for c in s:
        if c in mp:
            if not st or st.pop() != mp[c]: return False
        else: st.append(c)
    return not st
```

核心观察

每次 push 同时保存“到当前为止的最小值”，可用二元组 (value,currentMin) 或独立 minStack。

复杂度

$O(1)$ per op / $O(n)$

为什么这个算法成立

关键不变量: push(x,currentMin)。随后通过“pop 同步 → top 返回 value → getMin 返回当前 min”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

重复最小值需要正确处理，不能只在更小时入辅助栈后用错误条件弹出。

1. push(x,currentMin)



2. pop 同步

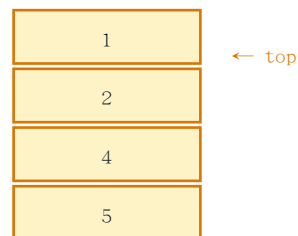


3. top 返回 value



4. getMin 返回当前 min

题目示意图



每个栈帧自带当时的 min，因此 pop 后自然恢复上一状态

扩展

MaxStack、单调结构

Go

```
type MinStack struct{st [][]int}
func Constructor() MinStack{return MinStack{}}
func(s *MinStack)Push(v int){m:=v;if len(s.st)>0&&s.st[len(s.st)-1][1]<m
{m=s.st[len(s.st)-1][1];s.st=append(s.st,[2]int{v,m})}
func(s *MinStack)Pop(){s.st=s.st[:len(s.st)-1]}
func(s *MinStack)Top()int{return s.st[len(s.st)-1][0]}
func(s *MinStack)GetMin()int{return s.st[len(s.st)-1][1]}
```

Python

```
class MinStack:
    def __init__(self):self.st=[]
    def push(self,val):self.st.append((val,min(val,self.st[-1][1] if sel
f.st else val)))
    def pop(self):self.st.pop()
    def top(self):return self.st[-1][0]
    def getMin(self):return self.st[-1][1]
```

核心观察

遇 `[` 时保存当前 `repeat` 和当前已构造字符串；遇 `]` 时弹出外层状态并拼接 `inner*repeat`。

复杂度

$O(\text{output size}) / O(n)$

为什么这个算法成立

关键不变量：解析数字 `k`。随后通过“遇 `[` 保存状态并清空 `inner` → 遇 `]` 恢复 `outer + k*inner` → 普通字符追加”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

多位数字如 `12[a]`；嵌套状态必须分别保存。

1. 解析数字 `k`



2. 遇 `[` 保存状态并清空 `inner`

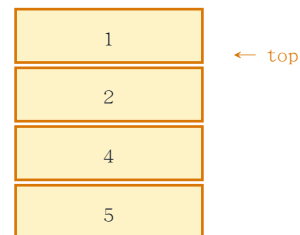


3. 遇 `]` 恢复 `outer + k*inner`



4. 普通字符追加

题目示意图



`3[a2[c]]`：栈先保存 `3` 和空串，再保存 `2` 和 `a`，内层完成后逐级还原

扩展

递归下降解析器、表达式求值

Go

```
func decodeString(s string) string { type frame struct{prev string;k int};st:=[]frame{};cur="";num:=0;for _,ch:=range s{if ch>='0'&&ch<='9'{num=num*10+int(ch-'0')}else if ch=='{'{st=append(st,frame{cur,num});cur="";num=0}else if ch=='}'{f:=st[len(st)-1];st=st[:len(st)-1];cur=f.prev+strings.Repeat(cur,f.k)}else{cur+=string(ch)}};return cur }
```

Python

```
def decodeString(s):
    st=[]; cur=''; num=0
    for ch in s:
        if ch.isdigit():num=num*10+int(ch)
        elif ch=='[':st.append((cur,num));cur='';num=0
        elif ch==']':prev,k=st.pop();cur=prev+cur*k
        else:cur+=ch
    return cur
```

核心观察

栈存还没找到更高温度的下标，并保持温度单调递减；当前温度更高时持续弹栈结算距离。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量：遍历 i, t 。随后通过“while $t > \text{temp}[\text{stackTop}] \rightarrow \text{pop } j$ 并 $\text{ans}[j] = i - j \rightarrow \text{push } i$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

存下标而不是值，因为答案是距离。

1. 遍历 i, t



2. while $t > \text{temp}[\text{stackTop}]$

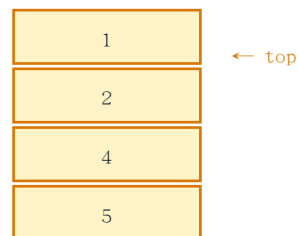


3. pop j 并 $\text{ans}[j] = i - j$



4. push i

题目示意图



栈内温度从底到顶递减；新高温会一次结算多个旧天

扩展

Next Greater Element、股票跨度

Go

```
func dailyTemperatures(t []int) []int { ans:=make([]int,len(t));st:=[]int{};for i,x:=range t{for len(st)>0&&t[st[len(st)-1]]<x{j:=st[len(st)-1];st=st[:len(st)-1];ans[j]=i-j};st=append(st,i)};return ans }
```

Python

```
def dailyTemperatures(t):  
    ans=[0]*len(t); st=[]  
    for i,x in enumerate(t):  
        while st and t[st[-1]]<x:  
            j=st.pop();ans[j]=i-j  
        st.append(i)  
    return ans
```

核心观察

维护高度单调递增的下标栈；遇更矮柱时，弹出的高度以当前 i 为右边界、弹栈后新栈顶为左边界。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量：加 0 哨兵。随后通过“保持递增栈 $\rightarrow$ 遇更矮时弹栈结算面积 $\rightarrow$ 当前下标入栈”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

推荐两端加 0 哨兵，统一清空逻辑；宽度= $i-left-1$ 。

1. 加 0 哨兵



2. 保持递增栈

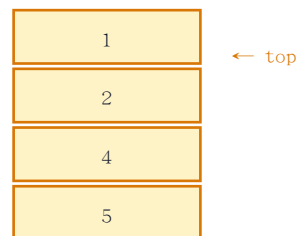


3. 遇更矮时弹栈结算面积



4. 当前下标入栈

题目示意图



弹出高度 h 时，它能向左右扩展到“第一个更矮柱”之间

扩展

最大矩形、单调栈边界题

Go

```
func largestRectangleArea(h []int) int { a:=append([]int{0},h...);a=append(a,0);st:=[]int{};ans:=0;for i,x:=range a{for len(st)>0&&a[st[len(st)-1]]>x{height:=a[st[len(st)-1]];st=st[:len(st)-1];w:=i-st[len(st)-1]-1;if height*w>ans{ans=height*w}};st=append(st,i)};return ans }
```

Python

```
def largestRectangleArea(heights):  
    a=[0]+heights+[0];st=[];ans=0  
    for i,h in enumerate(a):  
        while st and a[st[-1]]>h:  
            height=a[st.pop()]; width=i-st[-1]-1  
            ans=max(ans,height*width)  
        st.append(i)  
    return ans
```

核心观察

维护大小为 k 的小顶堆，堆顶始终是当前前 k 大元素中的最小值；扫描结束堆顶即第 k 大。

复杂度

$O(n \log k)$ / $O(k)$

为什么这个算法成立

关键不变量：逐个 `push`。随后通过“堆大小 $> k$ 就 `pop` → 始终保留最大 k 个 → 返回堆顶”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

第 k 大不是第 k 个不同值；也可 `QuickSelect` 平均 $O(n)$ 。

1. 逐个 `push`



2. 堆大小 $> k$ 就 `pop`

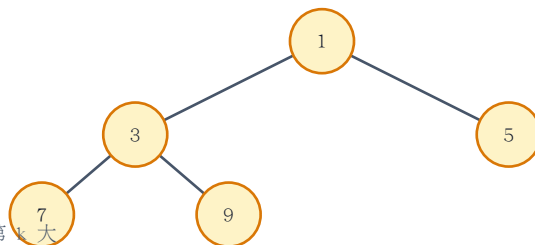


3. 始终保留最大 k 个



4. 返回堆顶

题目示意图



heap contains k largest seen so far; root 是这 k 个里最小的，也就是第 k 大

扩展

Top K 、`QuickSelect`

Go

```
func findKthLargest(nums []int, k int) int { h:=&IntHeap{};heap.Init(h);for _,x:=range nums{heap.Push(h,x);if h.Len()>k{heap.Pop(h)}};return (*h)[0] }
```

Python

```
import heapq
def findKthLargest(nums,k):
    h=[]
    for x in nums:
        heapq.heappush(h,x)
        if len(h)>k:heapq.heappop(h)
    return h[0]
```

核心观察

先统计 $\text{value} \rightarrow \text{freq}$ ，再维护大小 k 的 $(\text{freq}, \text{value})$ 小顶堆；或桶排序做到 $O(n)$ 。

复杂度

$O(n \log k)$ / $O(n)$

为什么这个算法成立

关键不变量：计频。随后通过“遍历频次表入堆 $\rightarrow$ 超过 k 弹最小频次 $\rightarrow$ 取堆中元素”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

Go 堆比较 freq ；Python tuple 自动按第一元素比较。

1. 计频



2. 遍历频次表入堆

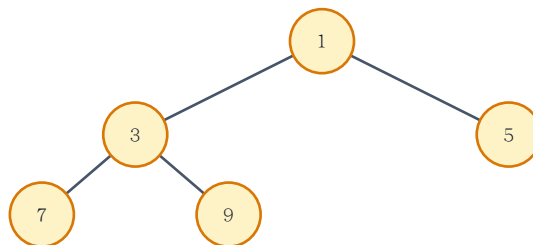


3. 超过 k 弹最小频次



4. 取堆中元素

题目示意图



堆只保留频率最大的 k 个 key

扩展

桶排序、Top K 流式统计

Go

```
func topKFrequent(nums []int, k int) []int { cnt:=map[int]int{};for _,x:=range nums{cnt[x]++};h:=&PairHeap{};heap.Init(h);for x,f:=range cnt{heap.Push(h,[2]int{f,x});if h.Len()>k{heap.Pop(h)}};ans=[]int{};for h.Len()>0{ans=append(ans,heap.Pop(h).([2]int)[1])};return ans }
```

Python

```
import heapq
def topKFrequent(nums,k):
    from collections import Counter
    h=[]
    for x,f in Counter(nums).items():
        heapq.heappush(h,(f,x))
        if len(h)>k:heapq.heappop(h)
    return [x for _,x in h]
```

核心观察

大顶堆 low 存较小一半，小顶堆 high 存较大一半；保持 len 差 ≤ 1 且 $\max(\text{low}) \leq \min(\text{high})$ 。

复杂度

add $O(\log n)$, median $O(1)$ / $O(n)$

为什么这个算法成立

关键不变量：先放到 low/high 合适一侧。随后通过“调整顺序约束 $\rightarrow$ 调整大小差 $\rightarrow$ 根据奇偶返回堆顶”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

Python 用负数模拟大顶堆；每次插入后要重新平衡。

1. 先放到 low/high 合适一侧



2. 调整顺序约束

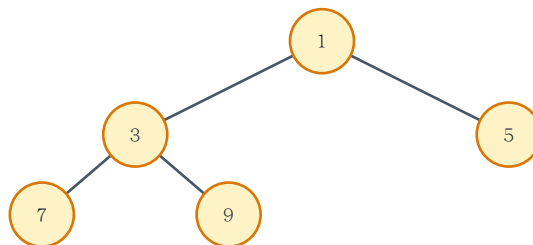


3. 调整大小差



4. 根据奇偶返回堆顶

题目示意图



low(max-heap) | median | high(min-heap), 两边始终有序且规模平衡

扩展

滑动窗口中位数、分位数数据结构

Go

```
type MedianFinder struct{low MaxHeap;high MinHeap}
func Constructor() MedianFinder{return MedianFinder{}}
func(m *MedianFinder)AddNum(x int){heap.Push(&m.low,x);heap.Push(&m.high,heap.Pop(&m.low).(int));if m.high.Len()>m.low.Len(){heap.Push(&m.low,heap.Pop(&m.high).(int)}}}
func(m *MedianFinder)FindMedian()float64{if m.low.Len()>m.high.Len(){return float64(m.low[0])};return float64(m.low[0]+m.high[0])/2}
```

Python

```
import heapq
class MedianFinder:
    def __init__(self):self.low=[];self.high=[]
    def addNum(self,num):
        heapq.heappush(self.low,-num)
        heapq.heappush(self.high,-heapq.heappop(self.low))
        if len(self.high)>len(self.low):heapq.heappush(self.low,-heapq.heappop(self.high))
    def findMedian(self):
        if len(self.low)>len(self.high):return -self.low[0]
        return (-self.low[0]+self.high[0])/2
```

核心观察

扫描价格，维护到当前为止最低买入价 `minPrice`，并用 `price-minPrice` 更新最大利润。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: `minPrice=inf`。随后通过“遍历 `price` $\rightarrow$ `profit=max(profit,price-minPrice)` $\rightarrow$ `minPrice=min(minPrice,price)`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

必须先买后卖，因此 `minPrice` 只来自当前之前/当前。

1. `minPrice=inf`



2. 遍历 `price`



3. `profit=max(profit,price-minPrice)`



4. `minPrice=min(minPrice,price)`

题目示意图

输入

状态/不变量

核心操作

结果

每一天假设今天卖出，最佳买点一定是此前最低价

扩展

多次交易、股票 DP

Go

```
func maxProfit(prices []int) int { low:=math.MaxInt;ans:=0;for _,p:=range prices{if p<low{low=p};if p-low>ans{ans=p-low}};return ans }
```

Python

```
def maxProfit(prices):  
    low=float('inf');ans=0  
    for p in prices:low=min(low,p);ans=max(ans,p-low)  
    return ans
```

核心观察

维护当前已知最远可达位置 far ；遍历到 i 时必须 $i \leq far$ ，再更新 $far = \max(far, i + \text{nums}[i])$ 。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量： $far=0$ 。随后通过“遍历 $i \leq far \rightarrow$ 更新 $far \rightarrow far \geq n-1$ 提前成功”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

一旦 $i > far$ 就说明出现不可跨越断点。

1. $far=0$



2. 遍历 $i \leq far$



3. 更新 far



4. $far \geq n-1$ 提前成功

题目示意图

输入

状态/不变量

核心操作

结果

far 表示当前所有选择能覆盖到的最右边界

扩展

Jump Game II、区间覆盖

Go

```
func canJump(nums []int) bool { far:=0;for i,x:=range nums{if i>far{return false};if i+x>far{far=i+x}};return true }
```

Python

```
def canJump(nums):  
    far=0  
    for i,x in enumerate(nums):  
        if i>far:return False  
        far=max(far,i+x)  
    return True
```

核心观察

按“层”扫描区间 $[0, \text{currentEnd}]$ ，在本层中计算 nextEnd ；走到 currentEnd 时必须跳一步并把边界更新为 nextEnd 。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量： $\text{curEnd}=0, \text{nextEnd}=0$ 。随后通过“扫描当前层 → 更新 nextEnd → 到 curEnd 时 $\text{steps}++$ 并扩层”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

最后一个位置不需要再跳，因此循环到 $n-2$ 。

1. $\text{curEnd}=0, \text{nextEnd}=0$



2. 扫描当前层



3. 更新 nextEnd



4. 到 curEnd 时 $\text{steps}++$ 并扩层

题目示意图

输入

状态/不变量

核心操作

结果

每一次 jump 相当于 BFS 的一层， nextEnd 是下一层能到达的最远点

扩展

BFS 层次视角、区间覆盖最少次数

Go

```
func jump(nums []int) int { steps,end,far:=0,0,0;for i:=0;i<len(nums)-1;i++{if i+nums[i]>far{far=i+nums[i]};if i==end{steps++;end=far}};return steps }
```

Python

```
def jump(nums):
    steps=end=far=0
    for i in range(len(nums)-1):
        far=max(far,i+nums[i])
        if i==end:steps+=1;end=far
    return steps
```

核心观察

先记录每个字符最后出现位置。扫描时 $end = \max(end, last[ch])$ ；当 $i == end$ 时当前段可以安全结束。

复杂度

$O(n)$ / $O(\text{字符集})$

为什么这个算法成立

关键不变量：记录 $last$ 。随后通过“ $start=0, end=0 \rightarrow$ 扫描并扩 $end \rightarrow i==end$ 时输出长度并开启新段”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

段结束条件是扫描位置追上该段所有字符的最远最后位置。

1. 记录 $last$



2. $start=0, end=0$



3. 扫描并扩 end



4. $i==end$ 时输出长度并开启新段

题目示意图

输入

状态/不变量

核心操作

结果

一段中出现过的所有字母，其最后位置必须都落在这段内部

扩展
区间合并、最小分段

Go

```
func partitionLabels(s string) []int { last:=[]int{};for i:=range s{last[s[i]-'a']=i};ans:=[]int{};start,end:=0,0;for i:=range s{if last[s[i]-'a']>end{end=last[s[i]-'a']};if i==end{ans=append(ans,end-start+1);start=i+1}};return ans }
```

Python

```
def partitionLabels(s):
    last={c:i for i,c in enumerate(s)};ans=[];start=end=0
    for i,c in enumerate(s):
        end=max(end,last[c])
        if i==end:ans.append(end-start+1);start=i+1
    return ans
```

核心观察

到第 i 阶只能从 $i-1$ 或 $i-2$ 来，因此 $f[i]=f[i-1]+f[i-2]$ ，可滚动变量压缩。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量： $a=1, b=1$ 。随后通过“循环 $i=2..n \rightarrow a, b=b, a+b \rightarrow$ 返回 b ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

$n=1/2$ 的初始化。

1. $a=1, b=1$



2. 循环 $i=2..n$



3. $a, b=b, a+b$



4. 返回 b

题目示意图

dp[2,0]	dp[2,1]	dp[2,2]	dp[2,3]	dp[2,4]	dp[2,5]
dp[1,0]	dp[1,1]	dp[1,2]	dp[1,3]	dp[1,4]	dp[1,5]
dp[0,0]	dp[0,1]	dp[0,2]	dp[0,3]	dp[0,4]	dp[0,5]

状态只依赖前两个，因此无需 dp 数组

扩展

斐波那契、不同步长组合

Go

```
func climbStairs(n int) int { a,b:=1,1;for i:=2;i<=n;i++{a,b=b,a+b};retu  
rn b }
```

Python

```
def climbStairs(n):  
    a=b=1  
    for _ in range(2,n+1):a,b=b,a+b  
    return b
```

核心观察

每行首尾为 1，中间 $cell = row-1[j-1] + row-1[j]$ 。

复杂度

$O(\text{numRows}^2)$ / $O(\text{output})$

为什么这个算法成立

关键不变量：逐行创建长度 $i+1$ 。随后通过“首尾设 1 → 填中间 → 追加到结果”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

每行长度 $i+1$ ；边界元素不要访问越界。

1. 逐行创建长度 $i+1$



2. 首尾设 1



3. 填中间



4. 追加到结果

题目示意图

dp[2,0]	dp[2,1]	dp[2,2]	dp[2,3]	dp[2,4]	dp[2,5]
dp[1,0]	dp[1,1]	dp[1,2]	dp[1,3]	dp[1,4]	dp[1,5]
dp[0,0]	dp[0,1]	dp[0,2]	dp[0,3]	dp[0,4]	dp[0,5]

上一行相邻两个数向下相加形成下一行中间值

扩展

组合数、滚动一维 Pascal

Go

```
func generate(n int) [][]int { ans:=[][]int{};for i:=0;i<n;i++){row:=make([]int,i+1);row[0],row[i]=1,1;for j:=1;j<i;j++){row[j]=ans[i-1][j-1]+ans[i-1][j]};ans=append(ans,row)};return ans }
```

Python

```
def generate(numRows):  
    ans=[]  
    for i in range(numRows):  
        row=[1]*(i+1)  
        for j in range(1,i):row[j]=ans[-1][j-1]+ans[-1][j]  
        ans.append(row)  
    return ans
```

核心观察

$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$, 选择“不抢当前”和“抢当前”两种互斥决策。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: $prev2=0, prev1=0$ 。随后通过“遍历 $x \rightarrow cur = \max(prev1, prev2+x) \rightarrow$ 滚动更新”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

状态含义要统一; 空数组/单元素边界。

1. $prev2=0, prev1=0$



2. 遍历 x



3. $cur = \max(prev1, prev2+x)$



4. 滚动更新

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

抢当前就不能抢前一间, 因此来自 $i-2$; 不抢则继承 $i-1$

扩展

House Robber II/III、选不相邻最大和

Go

```
func rob(nums []int) int { a,b:=0,0;for _,x:=range nums{if a+x>b{a,b=b,a+x}else{a,b=b,b}};return b }
```

Python

```
def rob(nums):  
    a=b=0  
    for x in nums:a,b=b,max(b,a+x)  
    return b
```

核心观察

完全背包: $dp[s]$ 表示组成和 s 的最少平方数个数; 枚举平方数后更新 $dp[s]=\min(dp[s], dp[s-q]+1)$ 。

复杂度

$O(n \cdot \sqrt{n})$ / $O(n)$

为什么这个算法成立

关键不变量: 生成 $squares$ 。随后通过“ $dp[0]=0 \rightarrow$ 遍历 s 或 $square \rightarrow$ 做 $\min$ 转移”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

初始化 $dp[0]=0$, 其余为大值; 也可 BFS 视作最短步数。

1. 生成 $squares$ 2. $dp[0]=0$ 3. 遍历 s 或 $square$ 4. 做 $\min$ 转移

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

把 $1, 4, 9, \dots$ 当成可无限使用的“硬币”, 求最少硬币数

扩展

完全背包、BFS 最短路、数论四平方定理

Go

```
func numSquares(n int) int { dp:=make([]int,n+1);for i:=1;i<=n;i++){dp[i]=n+1;for q:=1;q*q<=i;q++){if dp[i-q*q]+1<dp[i]{dp[i]=dp[i-q*q]+1}}};return dp[n] }
```

Python

```
def numSquares(n):  
    dp=[0]+[10**9]*n  
    for x in range(1,n+1):  
        q=1  
        while q*q<=x:  
            dp[x]=min(dp[x],dp[x-q*q]+1);q+=1  
    return dp[n]
```

核心观察

完全背包最小值: $dp[a] = \min(dp[a], dp[a - \text{coin}] + 1)$, 硬币可以无限使用。

复杂度

$O(\text{amount} * \#\text{coins}) / O(\text{amount})$

为什么这个算法成立

关键不变量: $dp[0] = 0$ 。随后通过“amount 逐步或 coin 外层 $\rightarrow$ 若 $a \geq \text{coin}$ 更新 $\rightarrow$ 返回 $dp[\text{amount}]$ 或 -1 ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

不可达状态使用 $\text{amount} + 1 / \text{inf}$; 最终要判是否仍不可达。

1. $dp[0] = 0$



2. amount 逐步或 coin 外层



3. 若 $a \geq \text{coin}$ 更新



4. 返回 $dp[\text{amount}]$ 或 -1

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

每个金额节点从 $a - \text{coin}$ 转移过来, 边权为 1

扩展

Coin Change II、完全背包

Go

```
func coinChange(coins []int, amount int) int { dp:=make([]int,amount+1);for i:=1;i<=amount;i++{dp[i]=amount+1;for _,c:=range coins{if c<=i&&dp[i-c]+1<dp[i]{dp[i]=dp[i-c]+1}}};if dp[amount]>amount{return -1};return dp[amount] }
```

Python

```
def coinChange(coins,amount):  
    dp=[0]+[amount+1]*amount  
    for a in range(1,amount+1):  
        for c in coins:  
            if c<=a:dp[a]=min(dp[a],dp[a-c]+1)  
    return -1 if dp[amount]>amount else dp[amount]
```

核心观察

$dp[i]$ 表示 $s[:i]$ 是否可被拆分；若存在 $j < i$ 且 $dp[j]$ && $s[j:i]$ 在字典，则 $dp[i]=true$ 。

复杂度

$O(n^2)$ / $O(n)$

为什么这个算法成立

关键不变量：wordSet。随后通过“ $dp[0]=true \rightarrow$ 枚举 $i, j \rightarrow$ 命中可拆分就置 $true$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

字典要 set 化；注意 dp 长度 $n+1$ 且 $dp[0]=true$ 。

1. wordSet

2. $dp[0]=true$ 3. 枚举 i, j 4. 命中可拆分就置 $true$

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

在字符串切分点形成 DAG， dp 判断是否存在从 0 到 n 的路径

扩展

Trie 优化、输出所有拆分方案

Go

```
func wordBreak(s string, dict []string) bool { st:=map[string]bool{};for _,w:=range dict{st[w]=true};dp:=make([]bool,len(s)+1);dp[0]=true;for i:=1;i<=len(s);i++){for j:=0;j<i;j++){if dp[j]&&st[s[j:i]]{dp[i]=true;break}}};return dp[len(s)] }
```

Python

```
def wordBreak(s,wordDict):  
    st=set(wordDict);dp=[False]*(len(s)+1);dp[0]=True  
    for i in range(1,len(s)+1):  
        dp[i]=any(dp[j] and s[j:i] in st for j in range(i))  
    return dp[-1]
```

核心观察

维护 `tails`: `tails[len-1]` 是长度为 `len` 的递增子序列可达到的最小结尾。对每个 `x` 用 `lower_bound` 替换/追加。

复杂度

$O(n \log n)$ / $O(n)$

为什么这个算法成立

关键不变量: `tails=[]`。随后通过“对 `x` 找 `first >= x` → 替换该位置或 `append` → `len(tails)` 即答案”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

`tails` 本身不一定是一条真实 LIS，只是各长度的最优尾值。

1. `tails=[]`



2. 对 `x` 找 `first >= x`



3. 替换该位置或 `append`



4. `len(tails)` 即答案

题目示意图

dp[2,0]	dp[2,1]	dp[2,2]	dp[2,3]	dp[2,4]	dp[2,5]
dp[1,0]	dp[1,1]	dp[1,2]	dp[1,3]	dp[1,4]	dp[1,5]
dp[0,0]	dp[0,1]	dp[0,2]	dp[0,3]	dp[0,4]	dp[0,5]

越小的尾值给未来留下越大增长空间，因此维护每个长度的最小可能尾值

扩展

恢复 LIS、二维 LIS/俄罗斯套娃

Go

```
func lengthOfLIS(nums []int) int { tails:=[]int{};for _,x:=range nums{i:=sort.SearchInts(tails,x);if i==len(tails){tails=append(tails,x)}else{tails[i]=x}};return len(tails) }
```

Python

```
from bisect import bisect_left
def lengthOfLIS(nums):
    tails=[]
    for x in nums:
        i=bisect_left(tails,x)
        if i==len(tails):tails.append(x)
        else:tails[i]=x
    return len(tails)
```

核心观察

负数会交换最大/最小角色，因此同时维护以当前位置结尾的 `maxProd` 和 `minProd`。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：初始化 `max=min=ans=nums[0]`。随后通过“对 `x` 保存旧 `max/min` → 更新 `max=max(x, x*oldMax, x*oldMin)` → 同理 `min`，更新 `ans`”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

遇负数时最大最小可能互换；0 会自然重置。

1. 初始化 `max=min=ans=nums[0]`



2. 对 `x` 保存旧 `max/min`



3. 更新 `max=max(x, x*oldMax, x*oldMin)`



4. 同理 `min`，更新 `ans`

题目示意图

dp[2,0]	dp[2,1]	dp[2,2]	dp[2,3]	dp[2,4]	dp[2,5]
dp[1,0]	dp[1,1]	dp[1,2]	dp[1,3]	dp[1,4]	dp[1,5]
dp[0,0]	dp[0,1]	dp[0,2]	dp[0,3]	dp[0,4]	dp[0,5]

负数 × 最小的积可能变成新的最大值，因此必须同时跟踪最小积

扩展

最大子数组和的非线性版本

Go

```
func maxProduct(nums []int) int { hi,lo,ans:=nums[0],nums[0],nums[0];for
_,x:=range nums[1:]{a,b:=hi,lo;hi=max3(x,a*x,b*x);lo=min3(x,a*x,b*x);if
hi>ans{ans=hi}};return ans }
```

Python

```
def maxProduct(nums):
    hi=lo=ans=nums[0]
    for x in nums[1:]:
        a,b=hi,lo
        hi=max(x,a*x,b*x);lo=min(x,a*x,b*x);ans=max(ans,hi)
    return ans
```

核心观察

总和若为奇数直接失败；转成 0/1 背包：能否选若干数使和为 $\text{total}/2$ 。

复杂度

$O(n \cdot \text{target}) / O(\text{target})$

为什么这个算法成立

关键不变量：算 $\text{total}/2$ 。随后通过 “ $\text{dp}[0]=\text{true} \rightarrow$ 对每个 x 倒序 $j \rightarrow \text{dp}[j] |= \text{dp}[j-x]$ ” 不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

0/1 背包容量必须倒序遍历，避免同一个元素被重复使用。

1. 算 $\text{total}/2$



2. $\text{dp}[0]=\text{true}$



3. 对每个 x 倒序 j



4. $\text{dp}[j] |= \text{dp}[j-x]$

题目示意图

$\text{dp}[2,0]$	$\text{dp}[2,1]$	$\text{dp}[2,2]$	$\text{dp}[2,3]$	$\text{dp}[2,4]$	$\text{dp}[2,5]$
$\text{dp}[1,0]$	$\text{dp}[1,1]$	$\text{dp}[1,2]$	$\text{dp}[1,3]$	$\text{dp}[1,4]$	$\text{dp}[1,5]$
$\text{dp}[0,0]$	$\text{dp}[0,1]$	$\text{dp}[0,2]$	$\text{dp}[0,3]$	$\text{dp}[0,4]$	$\text{dp}[0,5]$

把数组分成等和两组等价于寻找一个子集和为总和的一半

扩展

目标和、0/1 背包

Go

```
func canPartition(nums []int) bool { sum:=0;for _,x:=range nums{sum+=x};  
if sum%2==1{return false};t:=sum/2;dp:=make([]bool,t+1);dp[0]=true;for _  
,x:=range nums{for j:=t;j>=x;j--{dp[j]=dp[j]||dp[j-x]}};return dp[t] }
```

Python

```
def canPartition(nums):  
    s=sum(nums)  
    if s%2:return False  
    t=s//2;dp=[False]*(t+1);dp[0]=True  
    for x in nums:  
        for j in range(t,x-1,-1):dp[j]=dp[j] or dp[j-x]  
    return dp[t]
```

核心观察

$dp[i]$ 表示以 i 结尾的最长有效括号长度。仅当 $s[i]==")"$ 时，根据前一段长度定位可能匹配的左括号。

复杂度

$O(n)$ / $O(n)$

为什么这个算法成立

关键不变量：dp 全 0。随后通过“遇 $)$ 计算潜在匹配 $j \rightarrow$ 若 $s[j]==($ 则接上 $\rightarrow$ 加上 $j-1$ 之前的有效长度”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

索引 $j=i-dp[i-1]-1$ 的边界；也可用栈更直观。

1. dp 全 0



2. 遇 $)$ 计算潜在匹配 j



3. 若 $s[j]==($ 则接上



4. 加上 $j-1$ 之前的有效长度

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

... ([valid block]): 当前右括号可以跨过前一段有效块去寻找匹配左括号

扩展

括号 DP、栈解法

Go

```
func longestValidParentheses(s string) int { dp:=make([]int,len(s));ans:=0;for i:=1;i<len(s);i++{if s[i]==' '{j:=i-dp[i-1]-1;if j>=0&&s[j]==' '{ dp[i]=dp[i-1]+2;if j>0{dp[i]+=dp[j-1]};if dp[i]>ans{ans=dp[i]}}};return ans }
```

Python

```
def longestValidParentheses(s):
    dp=[0]*len(s);ans=0
    for i in range(1,len(s)):
        if s[i]==' ':
            j=i-dp[i-1]-1
            if j>=0 and s[j]=='(':
                dp[i]=dp[i-1]+2+(dp[j-1] if j>0 else 0)
            ans=max(ans,dp[i])
    return ans
```

核心观察

$dp[r][c] = dp[r-1][c] + dp[r][c-1]$; 可用一维数组滚动, $dp[c] += dp[c-1]$ 。

复杂度

$O(mn)$ / $O(n)$

为什么这个算法成立

关键不变量: dp 初始化全 1。随后通过“逐行遍历 $\rightarrow dp[c] = dp[c] + dp[c-1] \rightarrow$ 返回 $dp[n-1]$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

第一行第一列初始化为 1。

1. dp 初始化全 1



2. 逐行遍历



3. $dp[c] = dp[c] + dp[c-1]$



4. 返回 $dp[n-1]$

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

到每个格子只能从上或左, 因此路径数相加

扩展
组合数学、障碍物版本

Go

```
func uniquePaths(m,n int) int { dp:=make([]int,n);for i:=range dp{dp[i]=1};for r:=1;r<m;r++{for c:=1;c<n;c++{dp[c]+=dp[c-1]}};return dp[n-1] }
```

Python

```
def uniquePaths(m,n):  
    dp=[1]*n  
    for _ in range(1,m):  
        for c in range(1,n):dp[c]+=dp[c-1]  
    return dp[-1]
```

核心观察

$dp[r][c] = grid[r][c] + \min(top, left)$, 同样可滚动一维。

复杂度

$O(mn)$ / $O(n)$

为什么这个算法成立

关键不变量: 初始化首行/首列。随后通过“逐格取 $\min(top, left)$ → 加当前权重 → 返回右下角”不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

第一行/第一列只有单一来源, 需要单独初始化或用 `infinity` 哨兵。

1. 初始化首行/首列



2. 逐格取 $\min(top, left)$



3. 加当前权重



4. 返回右下角

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

每个格子的最优路径只依赖上方和左方两个已求出的子问题

扩展

网格最短路、三角形最小路径和

Go

```
func minPathSum(g [][]int) int { n:=len(g[0]);dp:=make([]int,n);for r:=0;r<len(g);r++{for c:=0;c<n;c++{if r==0&&c==0{dp[c]=g[r][c]}else if r==0{dp[c]=dp[c-1]+g[r][c]}else if c==0{dp[c]=dp[c]+g[r][c]}else{if dp[c-1]<dp[c]{dp[c]=dp[c-1]+g[r][c]}else{dp[c]=dp[c]+g[r][c]}}}};return dp[n-1] }
```

Python

```
def minPathSum(grid):
    m,n=len(grid),len(grid[0]);dp=[10**18]*n;dp[0]=0
    for r in range(m):
        for c in range(n):
            left=dp[c-1] if c else 10**18
            dp[c]=grid[r][c]+min(dp[c],left)
    return dp[-1]
```

核心观察

更实用的是中心扩展：枚举每个字符/字符间隙为中心，同时向两侧扩展，记录最长回文。

复杂度

$O(n^2)$ / $O(1)$

为什么这个算法成立

关键不变量：for center i 。随后通过“ $\text{expand}(i,i) \rightarrow \text{expand}(i,i+1) \rightarrow$ 记录最长区间”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

奇数中心和偶数中心都要检查。

1. for center i



2. $\text{expand}(i,i)$



3. $\text{expand}(i,i+1)$



4. 记录最长区间

题目示意图

$\text{dp}[2,0]$	$\text{dp}[2,1]$	$\text{dp}[2,2]$	$\text{dp}[2,3]$	$\text{dp}[2,4]$	$\text{dp}[2,5]$
$\text{dp}[1,0]$	$\text{dp}[1,1]$	$\text{dp}[1,2]$	$\text{dp}[1,3]$	$\text{dp}[1,4]$	$\text{dp}[1,5]$
$\text{dp}[0,0]$	$\text{dp}[0,1]$	$\text{dp}[0,2]$	$\text{dp}[0,3]$	$\text{dp}[0,4]$	$\text{dp}[0,5]$

回文关于中心对称；中心可能落在字符上，也可能落在两个字符之间

扩展

Manacher $O(n)$ 、回文 DP

Go

```
func longestPalindrome(s string) string { start,best:=0,0;expand:=func(l,r int)(int,int){for l>=0&&r<len(s)&&s[l]==s[r]{l--;r++;return l+1,r-l-1};for i:=0;i<len(s);i++){for _,p:=range [][]int{{i,i},{i,i+1}}{l,n:=expand(p[0],p[1]);if n>best{start,best=l,n}}};return s[start:start+best] }
```

Python

```
def longestPalindrome(s):
    start=0;best=0
    def expand(l,r):
        while l>=0 and r<len(s) and s[l]==s[r]:l-=1;r+=1
        return l+1,r-l-1
    for i in range(len(s)):
        for l,n in (expand(i,i),expand(i,i+1)):
            if n>best:start,best=l,n
    return s[start:start+best]
```

核心观察

$dp[i][j]$ 表示 $text1[:i]$ 与 $text2[:j]$ 的 LCS 长度；字符相等取 $dp[i-1][j-1]+1$ ，否则取上/左最大。

复杂度

$O(mn)$ / $O(mn)$ 可压缩到 $O(n)$

为什么这个算法成立

关键不变量：创建 $(m+1) \times (n+1)$ 。随后通过“双层循环 $\rightarrow$ 相等走对角+1 $\rightarrow$ 否则 $\max(\text{top}, \text{left})$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

子序列不要求连续；索引通常用 1-based dp 简化边界。

1. 创建 $(m+1) \times (n+1)$



2. 双层循环



3. 相等走对角+1



4. 否则 $\max(\text{top}, \text{left})$

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

二维表每格只依赖左、上、左上三个状态

扩展

编辑距离、diff、恢复 LCS

Go

```
func longestCommonSubsequence(a,b string) int { dp:=make([]int,len(b)+1)
;for i:=1;i<=len(a);i++){prev:=0;for j:=1;j<=len(b);j++){old:=dp[j];if a[i-1]==b[j-1]{dp[j]=prev+1}else if dp[j-1]>dp[j]{dp[j]=dp[j-1]};prev=old}}
;return dp[len(b)] }
```

Python

```
def longestCommonSubsequence(a,b):
    dp=[0]*(len(b)+1)
    for x in a:
        prev=0
        for j,y in enumerate(b,1):
            old=dp[j]
            dp[j]=prev+1 if x==y else max(dp[j],dp[j-1])
            prev=old
    return dp[-1]
```

核心观察

$dp[i][j]$ 是 $word1[:i]$ 变成 $word2[:j]$ 的最少操作。末字符相同走对角；不同则 $1+\min(\text{删、插、替换})$ 。

复杂度

$O(mn)$ / $O(mn)$

为什么这个算法成立

关键不变量：初始化边界。随后通过“遍历 $i, j \rightarrow$ 相同 $dp=diag \rightarrow$ 不同 $1+\min(\text{top, left, diag})$ ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

第一行/列初始化为 i/j ，代表与空串互转。

1. 初始化边界



2. 遍历 i, j



3. 相同 $dp=diag$



4. 不同 $1+\min(\text{top, left, diag})$

题目示意图

$dp[2,0]$	$dp[2,1]$	$dp[2,2]$	$dp[2,3]$	$dp[2,4]$	$dp[2,5]$
$dp[1,0]$	$dp[1,1]$	$dp[1,2]$	$dp[1,3]$	$dp[1,4]$	$dp[1,5]$
$dp[0,0]$	$dp[0,1]$	$dp[0,2]$	$dp[0,3]$	$dp[0,4]$	$dp[0,5]$

三种操作分别对应：删 $word1$ 字符(top)、插 $word2$ 字符(left)、替换(diag)

扩展

带权编辑距离、字符串相似度

Go

```
func minDistance(a,b string) int { dp:=make([]int,len(b)+1);for j:=range dp{dp[j]=j};for i:=1;i<=len(a);i++{nd:=make([]int,len(b)+1);nd[0]=i;for j:=1;j<=len(b);j++{if a[i-1]==b[j-1]{nd[j]=dp[j-1]}else{nd[j]=1+min3(dp[j],nd[j-1],dp[j-1])}};dp=nd};return dp[len(b)] }
```

Python

```
def minDistance(a,b):
    dp=list(range(len(b)+1))
    for i,x in enumerate(a,1):
        nd=[i]+[0]*len(b)
        for j,y in enumerate(b,1):
            nd[j]=dp[j-1] if x==y else 1+min(dp[j],nd[j-1],dp[j-1])
        dp=nd
    return dp[-1]
```

核心观察

利用 $x \oplus x = 0$ 、 $x \oplus 0 = x$ ，所有数字异或后成对元素抵消，只剩唯一元素。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量： $ans=0$ 。随后通过“for x : $ans \oplus x \rightarrow$ 成对值抵消 $\rightarrow$ 返回 ans ”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

只适用于“其他元素恰好出现两次”的设定。

1. $ans=0$



2. for x : $ans \oplus x$



3. 成对值抵消



4. 返回 ans

题目示意图

输入

状态/不变量

核心操作

结果

$a \oplus b \oplus a = b$ ，因为异或满足交换结合律且 $a \oplus a = 0$

扩展

Single Number II/III、位计数

Go

```
func singleNumber(nums []int) int { ans:=0;for _,x:=range nums{ans^=x};r
return ans }
```

Python

```
def singleNumber(nums):
    ans=0
    for x in nums:ans^=x
    return ans
```

核心观察

Boyer-Moore 投票：不同元素两两抵消；若多数元素出现超过 $n/2$ ，最终候选一定是它。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：count=0。随后通过“count==0 更新 candidate $\rightarrow$ 相同 +1 不同 -1 $\rightarrow$ 返回 candidate”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

若题目不保证一定存在多数元素，需要第二遍验证。

1. count=0



2. count==0 更新 candidate



3. 相同 +1 不同 -1



4. 返回 candidate

题目示意图

输入

状态/不变量

核心操作

结果

把多数元素与非多数元素配对抵消，多数元素因为数量优势最终仍会剩下

扩展

$n/3$ 多数元素扩展为两个候选

Go

```
func majorityElement(nums []int) int { cand,cnt:=0,0;for _,x:=range nums  
{if cnt==0{cand=x};if x==cand{cnt++}else{cnt--}};return cand }
```

Python

```
def majorityElement(nums):  
    cand=None;cnt=0  
    for x in nums:  
        if cnt==0:cand=x  
        cnt += 1 if x==cand else -1  
    return cand
```

核心观察

Dutch National Flag: 维护 $[0, low)$ 为 0, $[low, mid)$ 为 1, $[high, n)$ 为 2; mid 扫描未知区。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: $low=mid=0, high=n-1$ 。随后通过 “0: swap low/mid, 两者++ $\rightarrow$ 1: mid++ $\rightarrow$ 2: swap mid/high, high--” 不断缩小未解决空间或累积已知最优状态。面试时应重点说明: 每一步为什么不会漏解, 以及状态为什么足够表达后续决策。

易错点 / 边界

遇 2 与 high 交换后 mid 不能立即 ++, 因为换回来的元素还未分类。

1. $low=mid=0, high=n-1$



2. 0: swap low/mid, 两者++



3. 1: mid++



4. 2: swap mid/high, high--

题目示意图

输入

状态/不变量

核心操作

结果

0 区 | 1 区 | 未知区 | 2 区, 三个指针维护四段不变量

扩展

三路快排、三类原地分区

Go

```
func sortColors(nums []int) { l,i,r:=0,0,len(nums)-1;for i<=r{if nums[i]==0{nums[l],nums[i]=nums[i],nums[l];l++;i++}else if nums[i]==1{i++}else{nums[i],nums[r]=nums[r],nums[i];r--}} }
```

Python

```
def sortColors(nums):  
    l=i=0;r=len(nums)-1  
    while i<=r:  
        if nums[i]==0:nums[l],nums[i]=nums[i],nums[l];l+=1;i+=1  
        elif nums[i]==1:i+=1  
        else:nums[i],nums[r]=nums[r],nums[i];r-=1
```

核心观察

从右找第一对 $\text{nums}[i] < \text{nums}[i+1]$ 的下降前拐点；再从右找第一个 $> \text{nums}[i]$ 交换，最后反转后缀得到最小后缀。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量：从右找 pivot。随后通过“从右找 successor $\rightarrow$ swap $\rightarrow$ reverse suffix”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

若整个数组降序，答案是整体反转为最小排列。

1. 从右找 pivot



2. 从右找 successor



3. swap



4. reverse suffix

题目示意图

输入

状态/不变量

核心操作

结果

1 3 2: pivot=1, 右侧找最小更大值 2 交换得 2 3 1, 再反转后缀 $\rightarrow$ 2 1 3

扩展

字典序、prev permutation

Go

```
func nextPermutation(nums []int) { i:=len(nums)-2;for i>=0&&nums[i]>=nums[i+1]{i--};if i>=0{j:=len(nums)-1;for nums[j]<=nums[i]{j--};nums[i],nums[j]=nums[j],nums[i]};for l,r:=i+1,len(nums)-1;l<r;l,r=l+1,r-1{nums[l],nums[r]=nums[r],nums[l]} }
```

Python

```
def nextPermutation(nums):  
    i=len(nums)-2  
    while i>=0 and nums[i]>=nums[i+1]:i-=1  
    if i>=0:  
        j=len(nums)-1  
        while nums[j]<=nums[i]:j-=1  
        nums[i],nums[j]=nums[j],nums[i]  
    nums[i+1:]=reversed(nums[i+1:])
```

核心观察

把 `nums[i]` 看成 `next` 指针，数组形成带环链表；重复值就是环入口，使用 Floyd 两阶段定位。

复杂度

$O(n)$ / $O(1)$

为什么这个算法成立

关键不变量: `slow=nums[0], fast=nums[nums[0]]`。随后通过“先找相遇 $\rightarrow$ `ptr=0` $\rightarrow$ `ptr` 与 `slow` 同速直到相遇”不断缩小未解决空间或累积已知最优状态。面试时应重点说明：每一步为什么不会漏解，以及状态为什么足够表达后续决策。

易错点 / 边界

不能修改数组、不能用额外 `set` 时该建模最关键；索引/值范围保证指针合法。

1. `slow=nums[0], fast=nums[nums[0]]`



2. 先找相遇



3. `ptr=0`



4. `ptr` 与 `slow` 同速直到相遇

题目示意图

输入

状态/不变量

核心操作

结果

`index  $\rightarrow$  nums[index]` 构成函数图；两个不同索引指向同一重复值，形成环入口

扩展
环形链表 II、函数图

Go

```
func findDuplicate(nums []int) int { slow, fast:=nums[0],nums[nums[0]];for slow!=fast{slow=nums[slow];fast=nums[nums[fast]]};p:=0;for p!=slow{p=nums[p];slow=nums[slow]};return p }
```

Python

```
def findDuplicate(nums):  
    slow=nums[0];fast=nums[nums[0]]  
    while slow!=fast:slow=nums[slow];fast=nums[nums[fast]]  
    p=0  
    while p!=slow:p=nums[p];slow=nums[slow]  
    return p
```

60 天复习路径：一刷 → 二刷 → 语言互换 → 模拟面试

Day 01	#1 两数之和 / #49 字母异位词分组 / #128 最长连续序列	Day 21	#131 分割回文串 / #51 N 皇后
Day 02	#283 移动零 / #11 盛最多水的容器 / #15 三数之和	Day 22	#35 搜索插入位置 / #74 搜索二维矩阵
Day 03	#42 接雨水 / #3 无重复字符的最长子串 / #438 找到字符串中所有字母异位词	Day 23	#34 在排序数组中查找元素的第一个和最后一个位置 / #33 搜索旋转排序数组
Day 04	#560 和为 K 的子数组 / #239 滑动窗口最大值 / #76 最小覆盖子串	Day 24	#153 寻找旋转排序数组中的最小值 / #4 寻找两个正序数组的中位数
Day 05	#53 最大子数组和 / #56 合并区间 / #189 轮转数组	Day 25	#20 有效的括号 / #155 最小栈
Day 06	#238 除自身以外数组的乘积 / #41 缺失的第一个正数 / #73 矩阵置零	Day 26	#394 字符串解码 / #739 每日温度
Day 07	#54 螺旋矩阵 / #48 旋转图像 / #240 搜索二维矩阵 II	Day 27	#84 柱状图中最大的矩形 / #215 数组中的第 K 个最大元素
Day 08	#160 相交链表 / #206 反转链表 / #234 回文链表	Day 28	#347 前 K 个高频元素 / #295 数据流的中位数
Day 09	#141 环形链表 / #142 环形链表 II / #21 合并两个有序链表	Day 29	#121 买卖股票的最佳时机 / #55 跳跃游戏
Day 10	#2 两数相加 / #19 删除链表的倒数第 N 个结点 / #24 两两交换链表中的节点	Day 30	#45 跳跃游戏 II / #763 划分字母区间
Day 11	#25 K 个一组翻转链表 / #138 随机链表的复制 / #148 排序链表	Day 31	#70 爬楼梯 / #118 杨辉三角
Day 12	#23 合并 K 个升序链表 / #146 LRU 缓存 / #94 二叉树的中序遍历	Day 32	#198 打家劫舍 / #279 完全平方数
Day 13	#104 二叉树的最大深度 / #226 翻转二叉树 / #101 对称二叉树	Day 33	#322 零钱兑换 / #139 单词拆分
Day 14	#543 二叉树的直径 / #102 二叉树的层序遍历 / #108 将有序数组转换为二叉搜索树	Day 34	#300 最长递增子序列 / #152 乘积最大子数组
Day 15	#98 验证二叉搜索树 / #230 二叉搜索树中第 K 小的元素 / #199 二叉树的右视图	Day 35	#416 分割等和子集 / #32 最长有效括号
Day 16	#114 二叉树展开为链表 / #105 从前序与中序遍历序列构造二叉树 / #437 路径总和 III	Day 36	#62 不同路径 / #64 最小路径和
Day 17	#236 二叉树的最近公共祖先 / #124 二叉树中的最大路径和 / #200 岛屿数量	Day 37	#5 最长回文子串 / #1143 最长公共子序列
Day 18	#994 腐烂的橘子 / #207 课程表 / #208 实现 Trie (前缀树)	Day 38	#72 编辑距离 / #136 只出现一次的数字
Day 19	#46 全排列 / #78 子集 / #17 电话号码的字母组合	Day 39	#169 多数元素 / #75 颜色分类
Day 20	#39 组合总和 / #22 括号生成 / #79 单词搜索	Day 40	#31 下一个排列 / #287 寻找重复数

Day 41-60: 把“会做”升级成“能解释、能迁移、能双语实现”

Day 41-44

二刷哈希 / 双指针 / 滑窗 / 数组

闭卷说出不变量；每类任选 2 题用另一种语言重写。

Day 45-48

二刷链表 / 树

所有链表题先画指针图；树题先判断前序/后序/BFS。

Day 49-51

图 / 回溯 / 二分

强调 visited 时机、回溯撤销、lower_bound 边界。

Day 52-54

栈 / 堆 / 贪心

单调结构必须能解释“为什么被弹出的元素永远不会再成为候选”。

Day 55-57

DP 专项

每题先写状态定义、转移、初始化、遍历顺序，再写代码。

Day 58

Hard 集中复盘

42/76/239/25/23/124/51/4/84/295/32 等典型 Hard。

Day 59

模拟面试

3 轮 × 45 分钟：每轮 2 题；必须口述思路、复杂度、测试用例后再编码。

Day 60

最终闭卷

随机抽 20 题：60 秒分类，3 分钟讲解；再随机选 5 题 Go/Python 互换实现。

最终自测 Checklist

- ☐ 1. 能在 60 秒内把任意 Hot 100 题归类到一个核心 Pattern。
- ☐ 2. 能在写代码前先说出不变量 / dp 状态 / visited 语义 / 窗口合法条件。
- ☐ 3. 链表题可以不写代码先画完所有指针变化。
- ☐ 4. 二分题能统一写 lower_bound, 而不是每题重新发明边界。
- ☐ 5. 回溯题能明确 start / used / remain 的语义以及撤销点。
- ☐ 6. 树题能区分“向下携带状态”和“从子树向上返回信息”。
- ☐ 7. DP 题先写四件事: 状态、转移、初始化、遍历顺序。
- ☐ 8. Go 能熟练写 map/set、slice、queue、container/heap、递归闭包。
- ☐ 9. Python 能熟练写 dict/set、deque、heapq、bisect、Counter 与闭包 nonlocal。
- ☐ 10. 每道错题能归因: 模式没识别 / 不变量没想清 / 边界错误 / 语言细节 / 复杂度不达标。
- ☐ 11. 能把至少 30 道题在 Go 与 Python 之间互换实现。
- ☐ 12. Hard 题不是背答案: 能从朴素方案逐步推导到最优方案。